Database Administration and Management (DI-324)

Table Of Content

01	Advanced Data Models	Object-relational model, object-oriented model, concepts and examples.
02	File Organization Concepts	Records, files, storage methods, indexes and access techniques.
03	Transactional Processing	Transactions, ACID properties, transaction states and schedules.
04	Concurrency Control Techniques	Lost updates, locks, two-phase locking, timestamps and deadlocks.
05	Database Recovery Techniques	Log-based recovery, checkpoints, undo, redo and backup recovery.
06	Query Processing and Optimization	Query execution, query plans, indexes, cost estimation and optimization.
07	Database Programming	Stored procedures, functions, triggers, cursors and database connectivity concepts.
08	Database Integrity and Security	Constraints, authorization, authentication, SQL injection and access control.

Topic 1: Advanced Data Models

1.1 Introduction to Data Models

Definition: A data model is a collection of concepts, rules, and structures used to describe the organization of data, relationships among data elements, constraints on data, and operations permitted on the data in a database system.

In database design, a data model provides an abstract representation of real-world entities and their relationships.

Example: In a university database, the model describes entities such as Student, Course, and Instructor, along with relationships such as student enrollment in courses.

1.2 Why Are Data Models Important?

Data models are important because they:

1. Define how data is logically organized and represented.
2. Specify relationships and constraints between data elements.
3. Reduce ambiguity during database design.
4. Help maintain data integrity and consistency.
5. Provide a foundation for database implementation and querying.
6. Support efficient storage and retrieval according to application requirements.

1.3 Classification of Data Models

Database models can be classified into three broad categories.

Data Models

Conceptual

Entity-Relationship (ER) model

High-level entities and relationships

Logical

Relational model

Object-oriented model

Object-relational model

Physical

Files, pages, indexes, and storage structures

- Conceptual data model: Describes business entities and relationships without focusing on implementation.
- Logical data model: Defines the data structures and constraints independently of many physical storage details.
- Physical data model: Describes how data is actually stored and accessed in a particular database system.

The main focus of this course topic is on advanced logical data models, particularly the object-oriented and object-relational models.

1.4 Traditional Relational Data Model

Before studying advanced models, we need to understand the limitations of the traditional relational model.

Definition: The relational data model represents data as relations (tables), consisting of tuples (rows) and attributes (columns), with constraints that maintain data integrity.

Example:

STUDENT table

StudentID (PK)	Name	Department
101	Ali	IT
102	Sara	CS

Here, StudentID is the primary key.

The relational model is highly effective for structured business data. However, applications involving complex objects, multimedia, engineering designs, and nested structures may require additional modeling capabilities.

For example, a 3D engineering object might contain geometry, dimensions, materials, and operations. Representing all these properties in ordinary tables can require multiple related tables and complex joins.

This motivates advanced data models.

1.5 Object-Oriented Data Model (OODM)

Core concept · Advanced data model

Definition: The Object-Oriented Data Model represents data as objects that encapsulate state and behavior, using concepts such as classes, object identity, inheritance, encapsulation, and complex object structures.

Unlike a purely relational model, the object-oriented approach treats an object as a single conceptual unit containing both its data and associated operations.

A. Fundamental concepts

1. Object

An object is an identifiable instance containing state and behavior.

Example: A particular student with ID 101, name Ali, and an operation to display student details.

2. Class

A class defines the structure and behavior shared by its instances.

```
class Student {
   int studentId;
   String name;


   void display() {
      System.out.println(name);
   }
}
```

Here, Student defines a class, and its fields and method describe the structure and behavior of student objects.

3. Object identity (OID)

Object identity is a unique identity associated with an object independently of its current attribute values.

For example, two student objects might have identical names and other attributes but still represent two different students.

This differs from identifying an object solely through a primary key's value.

4. Encapsulation

Encapsulation combines data and the operations associated with that data into one unit and controls how the data is accessed.

For example, a bank account object may provide a withdraw() method that checks whether sufficient funds are available instead of allowing arbitrary changes to its balance.

5. Inheritance

Inheritance allows a class to acquire or extend the properties and behavior of another class.

Person

name, age

Student

studentID

Teacher

employeeID

Conceptual inheritance hierarchy

Both Student and Teacher inherit the common attributes of Person and can have their own specialized attributes.

6. Polymorphism

Polymorphism allows the same operation or interface to exhibit different behavior depending on the object's type.

For example, a calculateSalary() operation may work differently for a permanent employee and a contract employee.

7. Complex objects

An object may contain references to other objects, collections, or nested structures.

For example, a University object may contain departments, and each department may contain courses and instructors.

B. How does an object-oriented database work?

A typical object-oriented database management system stores and manages persistent objects, their identities, relationships, and supported operations.

Application

Creates or modifies objects

Object Database Management System

Object management, queries, transactions, persistence

Persistent Object Storage

Objects, identities, references and relationships

The exact architecture depends on the OODBMS implementation.

C. Advantages

- Naturally represents complex real-world objects.
- Supports encapsulation, inheritance, and polymorphism.
- Can reduce the need to transform complex objects into numerous relational tables.
- Useful for applications with complex interrelated data.
- Object identity can simplify the representation of object references.

D. Limitations

- There is no universally dominant standard query language comparable to SQL across all object databases.
- Interoperability and migration can be more difficult between different implementations.
- Complex object structures can make design and maintenance challenging.
- Relational databases may be more appropriate for conventional business reporting and tabular data.

E. Applications

- Computer-aided design (CAD) and computer-aided manufacturing (CAM).
- Multimedia and graphics systems.
- Scientific and engineering databases.
- Applications with deeply interconnected objects.

1.6 Object-Relational Data Model (ORDM)

Definition: The Object-Relational Data Model extends the relational model with selected object-oriented features, allowing relational tables and SQL-based operations to work with richer data types and structures.

The key idea is to preserve the relational foundation while supporting complex data representations.

A. Main features

1. User-defined types (UDTs)

A database designer can define a structured type for a particular domain.

For example, an address can have separate components such as street, city, and postal code.

2. Complex and structured attributes

A column may represent structured information rather than a single simple value, depending on the database system.

3. Collection types

Some systems support collections such as arrays or nested structures.

4. Methods and type behavior

Some object-relational systems support methods associated with user-defined types.

5. Inheritance

Certain systems support type or table inheritance, although the exact capabilities and behavior vary between products.

6. Relational foundation

Tables, rows, SQL queries, constraints, and relational operations remain central.

B. Example using PostgreSQL

Consider a university database that stores student addresses as structured values.

```
CREATE TYPE address_type AS (
    street VARCHAR(100),
    city VARCHAR(50),
    postal_code VARCHAR(15)
);


CREATE TABLE student (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    address address_type
);
```

This example illustrates a composite type used as a column in PostgreSQL.

Insert a record:

```
INSERT INTO student
```

```
VALUES (
   101,
   'Ali',
   ROW('Mall Road', 'Lahore', '54000')
);
```

Retrieve the city:

```
SELECT (address).city
FROM student
WHERE student_id = 101;
```

Explanation:

- CREATE TYPE defines a structured address type.
- address_type groups multiple address fields.
- The student table stores that structured value in one column.
- (address).city accesses the city component.

This is an example of adding structured types to a relational database; it does not mean that every relational DBMS supports the same features.

C. Advantages

- Retains relational tables and SQL capabilities.
- Supports richer data types and structured values.
- Can integrate complex information with existing relational applications.
- May reduce the need to split every complex structure across multiple tables.
- Useful when an application needs both relational querying and specialized data representations.

D. Limitations

- Advanced features vary among DBMS products.

- More complex types can increase schema and maintenance complexity.
- Specialized features may reduce portability between database systems.
- It does not automatically eliminate joins or guarantee better performance.

E. Applications

- Geographic information systems (GIS).
- Medical and scientific databases.
- Multimedia applications.
- Business systems that combine conventional records with complex structured data.

1.7 Object-Oriented vs. Object-Relational Data Models

Feature	Object-Oriented Model	Object-Relational Model
Fundamental representation	Objects	Relational tables with object-oriented extensions
Object identity	Usually explicit in the model	Relational keys remain important; some systems add object features
Encapsulation	Core concept	Supported selectively
Inheritance	Core modeling concept	Supported by some implementations
Querying	Varies by object database	SQL-based, with product-specific extensions
Existing relational systems	Requires an object-oriented database or suitable implementation	Extends relational database capabilities
Typical strength	Complex object relationships	Relational data plus complex structures

Exam point: An object-oriented database and an object-relational database are not the same thing. The former organizes persistent data around objects; the latter retains the relational foundation and adds object-oriented capabilities.

1.8 Relational vs. Object-Oriented vs. Object-Relational

Feature	Relational	Object-Oriented	Object-Relational
Basic structure	Tables	Objects	Tables plus richer types
Data representation	Rows and columns	State and behavior	Relational records and structured data
Identity	Usually primary keys	Object identity	Usually relational keys, with extensions
Complex objects	Often modeled across tables	Natural representation	Supported through extensions
Standard SQL	Yes, with implementation differences	Not universally the primary interface	Yes, with possible extensions

1.9 Important Long Questions for BS 7th Semester

1. Define advanced data models. Explain the Object-Oriented Data Model and its fundamental concepts.
2. Discuss object identity, encapsulation, inheritance, polymorphism, and complex objects with examples.
3. Explain the Object-Relational Data Model, its features, advantages, and limitations.
4. Compare the relational, object-oriented, and object-relational data models.
5. Discuss the applications of advanced data models in modern database systems.

6. Explain how object-oriented and object-relational models address the representation of complex data.

Quick revision

- Data model: Defines the structure, relationships, constraints, and operations of data.
- Object-oriented model: Represents data through objects, identity, and object-oriented concepts.
- Object-relational model: Combines the relational foundation with selected object-oriented features.
- OID: Identifies an object independently of its attribute values.
- Inheritance: Extends properties and behavior from one class or type to another.
- Encapsulation: Combines data with its associated operations.
- Polymorphism: Allows a common interface or operation to behave differently for different types.

Topic 2: File Organization Concepts

Course: Database Administration and Management (DI-324) · BS IT 7th Semester

2.1 Definition of File Organization

File organization is the method of arranging database records in secondary storage and managing their physical placement to support efficient retrieval, insertion, deletion, and updating of data.

At the database system level, file organization determines how records are placed into disk pages or blocks and how the DBMS locates the required records.

Example: A university database contains one million student records. The file organization method affects how efficiently the DBMS can find a student, insert a new record, or retrieve all students within a particular roll-number range.

2.2 Basic Concepts of Database Storage

Before studying file organization techniques, understand these fundamental terms.

1. Field (Attribute)

An individual data item, such as StudentID, Name, or GPA.

2. Record (Tuple)

A collection of related fields describing one entity.

Example: (101, 'Ali', 3.6).

3. File

A collection of related records stored and managed together.

4. Block / Page

A unit of data transferred between secondary storage and memory. Database systems commonly organize disk I/O around pages.

5. Index

An auxiliary data structure that helps the DBMS locate records without scanning the entire data file.

Storage hierarchy

Database File

Pages / Blocks

Records

Fields / Attributes

This is a conceptual hierarchy; actual storage layouts depend on the DBMS and its record format.

2.3 Types of File Organization

The principal methods covered here are:

1. Heap file organization
2. Sequential file organization
3. Hash file organization

4. Indexed file organization
5. Indexed sequential organization

These methods differ in record placement, search efficiency, insertion cost, and support for range queries.

2.4 Heap File Organization

Definition: Heap file organization stores records without maintaining an ordering based on a search key. New records are generally placed in available space, often using a free-space management mechanism.

Working

Suppose records are inserted in the following order:

Record position	StudentID	Name
1	105	Ali
2	101	Sara
3	110	Ahmed
4	103	Ayesha

The records are not sorted by StudentID.

To find student 103 without a suitable index, the DBMS may have to examine records until it finds the required one.

Search complexity

For a file containing \(n\) records, an unsuccessful search or search for a record near the end may require examining \(O(n)\) records.

If the target is equally likely to be anywhere in an unordered file, a simple sequential scan examines about \((n+1)/2\) records on average.

Advantages

- Simple storage and management.

- Efficient insertion when free space is readily available.
- Suitable for bulk loading and workloads dominated by inserts.

Limitations

- Searches may require a full scan.
- Records are not naturally available in sorted order.
- Deletion and space reuse require appropriate free-space management.

Real-world application: An append-heavy event log or a staging table where records are loaded first and analyzed later.

2.5 Sequential File Organization

Definition: Sequential file organization maintains records in a specified order, commonly according to a search key.

Example:

Position	StudentID	Name
1	101	Sara
2	103	Ayesha
3	105	Ali
4	110	Ahmed

Records are ordered by StudentID.

Searching

If the records are sorted and stored in a structure that supports random access, binary search can find a record in \(O(\log n)\) comparisons. If the file is stored on disk and only supports sequential access, the practical I/O cost differs; a block index or other access structure may be needed.

Advantages

- Efficient sequential and ordered processing.

- Suitable for reports and batch-processing workloads.
- Can support range queries effectively when records are maintained in key order.

Limitations

- Maintaining order during insertion can be expensive.
- Insertions and deletions may require record movement or overflow handling.
- Frequent modifications can make maintenance costly.

Real-world application: Processing student examination records in roll-number order to generate result sheets.

2.6 Hash File Organization

Definition: Hash file organization uses a hash function to map a search-key value to a bucket or storage location.

A hash function can be represented as:

\[h(k) = \text{bucket address} \]

For example, suppose:

\[h(k)=k\bmod 10 \]

For Student ID 105:

\[h(105)=105\bmod 10=5 \]

Therefore, the record is assigned to bucket 5.

Search key: StudentID = 105

Hash function

h(105) = 5

Bucket 5

Record location

Hash collisions

A collision occurs when different keys map to the same bucket.

For example:

\[h(105)=5,\quad h(115)=5 \]

Both keys map to bucket 5.

Common collision-resolution techniques include:

- Separate chaining / overflow chaining: Store colliding entries in an associated chain or overflow structure.
- Open addressing: Probe alternative positions in the hash table. This is more common in in-memory hash tables than in conventional disk-based DBMS file organizations.

Dynamic hashing techniques, such as extendible hashing and linear hashing, can adapt bucket structures as the file grows.

Complexity

Hash-based exact-match lookup is often \(O(1)\) on average under suitable assumptions, but collisions, overflow chains, and poor hash distribution can increase the cost. This complexity does not directly describe disk I/O in every DBMS.

Advantages

- Efficient exact-key lookups.
- Often suitable for equality-based searches.
- Can provide efficient insertion when bucket capacity and overflow are managed well.

Limitations

- Poor for ordered range queries.
- Collisions can reduce performance.
- Hash structure resizing and overflow management can be complex.

Real-world application: Looking up a customer account by an exact account number.

2.7 Indexed File Organization

Definition: Indexed file organization uses an auxiliary search structure containing key values and pointers or references to records or data pages.

The index reduces the need to scan the complete data file.

Example

Suppose a file contains thousands of student records. An index on StudentID allows the DBMS to locate the page containing student 105.

Index entries

101 → Page 1

105 → Page 3

110 → Page 5

The index directs the search to the relevant page.

Data pages

Page 1 · Page 2 · Page 3 · Page 4 · Page 5

The arrows in this example are illustrative. Actual indexes may use record identifiers, page identifiers, or other implementation-specific references.

Types of indexes

1. Primary index: In the classical file-organization terminology, an index built on the ordering key of an ordered data file. The term can have different meanings in practical DBMS products.

2. Clustering index: An index associated with a field that determines the physical ordering of records, with multiple records potentially sharing a value.

3. Secondary index: An additional access path on a field that does not determine the file's physical ordering.

4. Dense index: Contains an index entry for every search-key value or record, depending on its design.

5. Sparse index: Contains entries for selected search-key values, often one entry per data block or group of records.

6. Multilevel index: Uses an index over another index to reduce the number of search steps in large files.

Advantages

- Faster record retrieval.
- Supports efficient equality and, with appropriate index structures, range queries.
- Can reduce disk page accesses substantially.

Limitations

- Requires additional storage.
- Inserts, updates, and deletes may require index maintenance.
- Too many indexes can increase write costs.
- Index effectiveness depends on selectivity, data distribution, and query patterns.

Real-world application: An index on StudentID helps the DBMS find a student's record without scanning the entire student table.

2.8 B-Tree and B+ Tree Index Structures

These are particularly important for university-level database study.

B-Tree

A B-tree is a balanced search-tree structure in which keys and associated data references may appear in both internal and leaf nodes, depending on the implementation.

B+ Tree

A B+ tree is a balanced multiway search tree in which data-entry references are stored at the leaf level, while internal nodes guide searches. Leaf nodes are commonly linked to support ordered traversal and range queries.

Conceptual B+ tree

Root

50 | 100

Keys below 50

Keys 50–99

Keys 100 and above

Simplified illustration: actual tree fan-out and separator-key conventions vary.

Why are B+ trees useful?

- They remain balanced as records are inserted and deleted.
- Their high fan-out reduces tree height.
- They support efficient exact-match searches.
- Linked leaves make range scans efficient.
- They are widely used in database indexes, including many relational DBMS implementations.

Complexity

For a balanced tree containing \(n\) indexed keys, search, insertion, and deletion are typically \(O(\log n)\) in tree operations. Actual database performance also depends on page I/O, cache behavior, and index maintenance.

2.9 Indexed Sequential Organization

Definition: Indexed sequential organization stores records in sequential key order and maintains an index to support direct access to appropriate data blocks.

It combines the ordered access of sequential files with index-based searching.

Advantages

- Supports both sequential processing and direct lookup.
- Can be effective for ordered and range-based retrieval.
- Allows an index to guide the search to the correct region of the file.

Limitations

- Index storage and maintenance are required.
- Insertions may use overflow areas or require reorganization.
- More complicated than a simple heap file.

Example: A legacy banking system processes account records sequentially for daily reporting and uses an index to find individual accounts.

2.10 Comparison of File Organization Methods

Property	Heap	Sequential	Hash	Indexed / B+ Tree
Record order	Unordered	Sorted by key	Determined by hashing	Depends on data-file layout
Exact search	Often \(O(n)\) scan	Depends on access method	Often efficient on average	Efficient
Range queries	Usually costly without index	Good when ordered	Poor	Good with ordered index
Insertion	Usually simple	May be costly	Bucket-dependent	Requires index maintenance
Extra storage	Minimal organization overhead	Depends on implementation	Buckets/overflow may be needed	Index storage required
Typical use	Insert-heavy files	Ordered processing	Exact-match lookup	General-purpose searching

The complexities in this table are general descriptions rather than guarantees; database implementations and workload characteristics matter.

2.11 Factors Affecting File Organization Selection

When selecting a file organization, a database administrator considers:

1. Search frequency: How often records must be retrieved.
2. Insertion and update frequency: How often data changes.
3. Range queries: Whether users search for intervals of values.
4. Storage overhead: How much space indexes or overflow structures require.
5. Disk I/O cost: How many page reads and writes are expected.
6. Data growth: Whether the structure can handle an expanding file efficiently.
7. Workload requirements: Whether the application prioritizes fast lookups, sequential reports, or frequent updates.

Important exam questions

1. Define file organization and explain its role in physical database design.
2. Explain heap, sequential, and hash file organizations with examples.
3. What is a hash collision? Explain collision-resolution techniques.
4. Define database indexing and explain dense, sparse, primary, and secondary indexes.
5. Differentiate between B-tree and B+ tree index structures.
6. Explain indexed sequential organization and compare it with heap organization.
7. Discuss the factors affecting the selection of a file organization method.
8. Explain how file organization affects disk I/O and database performance.

Quick revision

- Heap: Records are not ordered by a search key.

- Sequential: Records are maintained in a specified order.
- Hashing: A hash function maps a key to a bucket.
- Index: An auxiliary structure helps locate records.
- B+ tree: A balanced index structure well suited to ordered and range searches.
- Indexed sequential: Combines ordered storage with an index.

Topic 03: Transactional Processing

Complete notes in English | University level | Definitions, explanations, examples, SQL, and exam preparation

1. Introduction to Transactional Processing

Definition

Transactional processing is the process of executing a group of database operations as a single logical unit of work called a transaction. It ensures that database operations are performed reliably, accurately, and consistently, even when errors, system failures, or concurrent users are involved.

A transaction may include one or more operations, such as:

- Reading data from a database.
- Inserting new records.
- Updating existing records.
- Deleting records.
- Transferring money between bank accounts.

Real-world example: Bank transfer

Suppose Ali transfers Rs. 5,000 from his bank account to Ahmed's account.

Initially:

- Ali's balance = Rs. 20,000
- Ahmed's balance = Rs. 10,000

The transaction consists of two main operations:

1. Deduct Rs. 5,000 from Ali's account.
2. Add Rs. 5,000 to Ahmed's account.

After a successful transaction:

- Ali's balance = Rs. 15,000
- Ahmed's balance = Rs. 15,000

The total amount remains Rs. 30,000.

However, suppose the system deducts Rs. 5,000 from Ali's account but crashes before adding the amount to Ahmed's account. The database becomes inconsistent.

Transactional processing prevents this problem by ensuring that either both operations succeed or neither operation takes effect.

2. What Is a Transaction?

Definition

A transaction is a sequence of database operations treated as one indivisible logical unit of work. It must satisfy the ACID properties to maintain data integrity and reliability.

A transaction generally begins with a BEGIN or START TRANSACTION statement and ends with either COMMIT or ROLLBACK.

Example using SQL

Consider two bank accounts stored in a table named Accounts.

START TRANSACTION;

UPDATE Accounts

SET balance = balance - 5000

WHERE account_id = 101;

```
UPDATE Accounts
SET balance = balance + 5000
WHERE account_id = 102;


COMMIT;
```

Explanation

- START TRANSACTION: Begins a new transaction.
- First UPDATE: Deducts Rs. 5,000 from account 101.
- Second UPDATE: Adds Rs. 5,000 to account 102.
- COMMIT: Makes the transaction's changes permanent.

If an error occurs before the transaction commits, the application or database can roll back the transaction so that the incomplete transfer does not remain in the database.

```
ROLLBACK;
```

ROLLBACK cancels the transaction's uncommitted changes, subject to the database system's transaction semantics.

Note: Exact syntax and behavior can vary between database systems. For example, MySQL requires a transactional storage engine such as InnoDB for this behavior.

3. Types of Database Operations

Transactions commonly use the following operations.

Operation	Meaning	Example
READ(X)	Reads the value of data item X	Read account balance
WRITE(X)	Changes the value of data item X	Update balance
INSERT	Adds a new record	Register a new student

UPDATE	Modifies an existing record	Change student marks
DELETE	Removes a record	Delete an inactive account
COMMIT	Permanently commits transaction changes	Complete a bank transfer
ROLLBACK	Cancels uncommitted changes	Cancel an incomplete transfer

In transaction theory, READ(X) and WRITE(X) are abstract operations used to study transaction behavior. SQL statements implement these operations in actual database systems.

4. ACID Properties

ACID is the foundation of reliable transaction processing.

ACID stands for:

- A — Atomicity
- C — Consistency
- I — Isolation
- D — Durability

These four properties ensure that transactions are executed safely and that the database remains reliable.

1. Atomicity

A transaction is treated as a single indivisible unit. Either all its operations succeed or none of its changes are applied.

Example: Both sides of a bank transfer must succeed, or the transfer must be cancelled.

2. Consistency

A transaction must preserve database rules and constraints, moving the database from one valid state to another valid state.

Example: A bank transfer should preserve the total money across both accounts and satisfy balance constraints.

3. Isolation

Concurrent transactions should not interfere in ways that violate the database's isolation guarantees.

Example: Two users booking the same seat must not both successfully reserve it if the seat is unique.

4. Durability

Once a transaction is successfully committed, its changes must survive subsequent system crashes or restarts, subject to the database's durability guarantees.

Example: After a successful bank transfer, the updated balances remain recorded even if the server restarts.

4.1 Atomicity in Detail

Definition

Atomicity means all or nothing. A transaction cannot leave only a portion of its intended changes applied when it fails.

Example

A university system processes a student's admission by performing three operations:

1. Create a student record.
2. Assign a registration number.
3. Record the semester enrollment.

If the system fails after creating the student record but before recording enrollment, atomicity requires the transaction to be rolled back so that the incomplete admission is not left behind.

How is atomicity achieved?

Database systems commonly use:

- Transaction logs.
- Undo information.
- Rollback mechanisms.
- Recovery procedures.

The DBMS records enough information to reverse uncommitted changes after a failure.

4.2 Consistency in Detail

Definition

Consistency ensures that every committed transaction preserves the database's defined integrity constraints, relationships, and business rules.

Example: Account balance constraint

Suppose a bank defines the following rule:

\[\text{balance} \geq 0 \]

A transaction attempting to withdraw Rs. 30,000 from an account containing only Rs. 20,000 must be rejected if overdrafts are prohibited.

```
CREATE TABLE Accounts (
   account_id INT PRIMARY KEY,
   balance DECIMAL(12, 2)
      CHECK (balance >= 0)
);
```

The CHECK constraint expresses the rule, although enforcement details can vary across database products and versions.

Other consistency rules

- Every student must have a unique registration number.
- Every order must reference an existing customer.

- An employee's salary must follow the organization's permitted rules.
- A product's quantity should not become negative if negative inventory is prohibited.

Important: The DBMS enforces defined constraints, but the application designer must define appropriate business rules. Consistency does not automatically mean that every business requirement has been captured.

4.3 Isolation in Detail

Definition

Isolation controls how concurrent transactions interact and what changes they are allowed to observe.

Without suitable isolation, concurrent transactions may cause incorrect results.

Example: Lost update

Suppose a bank account has Rs. 10,000.

Two transactions execute at nearly the same time:

- Transaction T1 withdraws Rs. 2,000.
- Transaction T2 withdraws Rs. 3,000.

Both transactions read the original balance of Rs. 10,000.

T1 calculates:

\[10000-2000=8000 \]

T2 calculates:

\[10000-3000=7000 \]

If T1 writes Rs. 8,000 and T2 subsequently overwrites it with Rs. 7,000, the final balance is Rs. 7,000 instead of the correct Rs. 5,000.

This is a lost update problem.

Isolation techniques, such as locking, serializable execution, or appropriate concurrency-control mechanisms, can prevent this anomaly.

Different isolation levels provide different guarantees. Not every transaction requires the strongest isolation level.

4.4 Durability in Detail

Definition

Durability means that once a transaction commits successfully, its changes are preserved despite subsequent failures covered by the database's durability guarantees.

Example

A student pays a university fee of Rs. 25,000 online.

The database confirms the payment and commits the transaction. If the database server crashes immediately afterward, the payment record must still be recoverable when the system restarts.

How is durability achieved?

Database systems commonly use:

- Write-ahead logging (WAL).
- Persistent transaction logs.
- Disk storage and appropriate synchronization.
- Checkpoints and crash-recovery procedures.
- Replication or other redundancy mechanisms where configured.

The exact protection depends on the database configuration and the failures it is designed to tolerate.

5. Difference Between ACID Properties

Property	Main purpose	Question it answers
Atomicity	Prevents partial transactions	Did all operations succeed or none?
Consistency	Preserves database rules	Is the resulting database state valid?

Isolation	Controls concurrent access	Can concurrent transactions interfere incorrectly?
Durability	Preserves committed changes	Will committed data survive a failure?

Easy way to remember

- Atomicity = All or nothing.
- Consistency = Rules remain valid.
- Isolation = Safe concurrency.
- Durability = Changes remain saved.

6. Transaction States

A transaction passes through different states during execution. Understanding these states helps explain how a DBMS handles successful transactions and failures.

The main transaction states are:

1. Active
2. Partially committed
3. Committed
4. Failed
5. Aborted
6. Terminated

6.1 Active State

Definition: The active state is the initial state in which a transaction is executing its read and write operations.

Example:

START TRANSACTION;

UPDATE Accounts

SET balance = balance - 5000

WHERE account_id = 101;

The transaction is active while it performs its operations and has not yet committed or aborted.

6.2 Partially Committed State

Definition: A transaction enters the partially committed state after executing its final statement but before the DBMS has completed the work required to make the transaction's commit successful and durable.

Example:

A bank transfer has completed both balance updates, and the DBMS is processing the commit.

The transaction has performed its logical work, but a failure can still prevent a successful commit.

6.3 Committed State

Definition: A transaction enters the committed state when the DBMS has successfully completed the commit according to its guarantees.

Example:

The DBMS confirms the bank transfer and makes its committed result durable.

After a successful commit, a normal rollback cannot simply cancel the transaction as though it had never happened. A compensating transaction may be used to reverse the business effect.

6.4 Failed State

Definition: A transaction enters the failed state when it can no longer proceed normally because of an error or failure.

Possible causes include:

- Power failure.

- Database server crash.
- Constraint violation.
- Deadlock victim selection.
- Invalid data or application error.
- Disk or storage failure.

Example:

A transaction attempts to insert a duplicate value into a primary-key column, and the DBMS rejects the operation.

Depending on the error and DBMS, the statement or the entire transaction may fail.

6.5 Aborted State

Definition: A transaction enters the aborted state after the DBMS has rolled back its effects or otherwise restored the transaction's changes to an appropriate state.

Example:

A bank transfer fails, so the DBMS undoes the uncommitted deduction from the source account.

An aborted transaction may be restarted as a new attempt, or it may end permanently.

6.6 Terminated State

Definition: A transaction reaches the terminated state after it has completed its lifecycle and the DBMS has finished the necessary transaction cleanup.

This can happen after either a successful commit or an abort.

Transaction state diagram

Active

Partially Committed

Committed

Terminated

Failure paths: Active or Partially Committed → Failed → Aborted → Terminated.

The diagram shows a simplified lifecycle. In an actual DBMS, transaction-state transitions and cleanup details may vary.

7. Transaction Schedules

7.1 Definition of a Schedule

A transaction schedule is the sequence in which the operations of one or more transactions are executed by a database system.

A schedule must preserve the original order of operations within each individual transaction, although operations from different transactions may be interleaved.

Schedules are important because several users may access and modify the database simultaneously.

Example: Two transactions

Let the initial value of account X be 100.

Transaction T1:

READ(X)

X = X - 20

WRITE(X)

Transaction T2:

READ(X)

X = X + 10

WRITE(X)

The transactions may execute in different orders.

7.2 Serial Schedule

Definition

A serial schedule executes one transaction completely before starting another transaction.

There is no interleaving between the operations of different transactions.

Example:

Serial Schedule S1

T1

T2

Execution order: T1 completes all its operations, then T2 completes all its operations.

T1: READ(X)

T1: WRITE(X)

T2: READ(X)

T2: WRITE(X)

Advantages:

- Simple to understand.
- Avoids interference between concurrent transactions.
- Provides a useful correctness reference.

Disadvantages:

- Reduces concurrency.
- May leave database resources underutilized.
- Can reduce throughput when transactions must wait unnecessarily.

7.3 Non-Serial Schedule

Definition

A non-serial schedule interleaves operations from multiple transactions rather than executing each transaction completely in isolation.

Example:

T1: READ(X)

T2: READ(X)

T1: WRITE(X)

T2: WRITE(X)

Advantages:

- Allows concurrent work.
- Can improve throughput.
- Can improve resource utilization.

Disadvantages:

- May produce incorrect results.
- Can cause lost updates or other anomalies if concurrency is not controlled.
- Requires appropriate concurrency-control mechanisms.

A non-serial schedule is not necessarily incorrect. It can be safe if it is equivalent to a correct serial schedule under the relevant correctness criteria.

7.4 Serializable Schedule

Definition

A serializable schedule is a schedule whose overall effect is equivalent to that of some serial execution of the same transactions.

Serializability is a major correctness criterion for concurrent transaction processing.

There are two important forms:

- Conflict serializability.
- View serializability.

Conflict serializability

Two operations conflict when they belong to different transactions, access the same data item, and at least one operation is a write.

The conflicting operation pairs are:

- Read–Write.
- Write–Read.
- Write–Write.

Read–Read operations do not conflict under the standard conflict-serializability model.

A schedule is conflict-serializable if it can be transformed into a serial schedule by swapping adjacent non-conflicting operations.

Example

Consider:

T1: READ(X)

T1: WRITE(X)

T2: READ(X)

T2: WRITE(X)

This is equivalent to executing T1 completely before T2, so it is conflict-serializable.

Now consider the interleaved schedule:

T1: READ(X)

T2: READ(X)

T1: WRITE(X)

T2: WRITE(X)

The conflict order is:

- T2's READ(X) occurs before T1's WRITE(X), giving T2 → T1.
- T1's WRITE(X) occurs before T2's WRITE(X), giving T1 → T2.

These dependencies form a cycle. Therefore, this schedule is not conflict-serializable.

7.5 Precedence Graph

A precedence graph, also called a serialization graph, is used to test conflict serializability.

- Each transaction is represented by a node.
- An edge from T1 to T2 means a conflicting operation of T1 occurs before an operation of T2.
- A cycle indicates that the schedule is not conflict-serializable.
- An acyclic graph indicates that the schedule is conflict-serializable.

Example: Acyclic graph

T1

T2

No cycle: the schedule is conflict-serializable, with T1 before T2.

Example: Cyclic graph

T1

T2

T2 → T1 creates a cycle

Cycle present: the schedule is not conflict-serializable.

8. Transaction Processing Example in SQL

Consider a university database with a Students table.

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100),
    fee_paid DECIMAL(10, 2)
```

);

Suppose a student pays a fee of Rs. 10,000.

START TRANSACTION;

UPDATE Students

SET fee_paid = fee_paid + 10000

WHERE student_id = 101;

COMMIT;

If the payment operation cannot be completed and the transaction remains uncommitted, the application may roll it back:

ROLLBACK;

Important practical point: A real payment system should also validate the student, verify the payment with the payment provider, and use a unique payment reference to prevent duplicate payments. Transaction processing alone does not prevent every application-level error.

9. Advantages of Transactional Processing

1. Data integrity: Helps maintain correct and consistent data.
2. Reliability: Handles transaction failures through rollback and recovery.
3. Concurrency support: Allows multiple users to work with the database safely.
4. Error handling: Prevents incomplete operations from being treated as successful transactions.
5. Crash recovery: Helps restore the database after unexpected failures.
6. Business correctness: Supports reliable banking, shopping, reservation, and university systems.

10. Limitations and Challenges

1. Performance overhead: Logging, locking, and recovery operations require resources.
2. Deadlocks: Transactions may wait for resources held by one another.
3. Contention: Concurrent transactions may have to wait for access to the same data.
4. Complexity: Correct transaction design requires careful handling of failures and concurrency.
5. Distributed transactions: Maintaining atomicity across multiple services or databases can be difficult.
6. Long-running transactions: Transactions that hold resources for extended periods can reduce system performance.

11. Important Exam Questions

These questions are suitable for BS Computer Science, 7th Semester DBMS preparation.

Short questions

1. Define transactional processing.
2. What is a database transaction?
3. What does ACID stand for?
4. Define atomicity with an example.
5. Explain consistency in DBMS.
6. What is transaction isolation?
7. Define durability.
8. What is the difference between COMMIT and ROLLBACK?
9. Define serial and non-serial schedules.
10. What is conflict serializability?

11. What is a precedence graph?
12. Name the major transaction states.

Long questions

1. Explain transactional processing and the ACID properties with real-world examples.
2. Discuss the different transaction states with a diagram.
3. Explain serial, non-serial, and serializable schedules with examples.
4. Define conflict serializability and explain how a precedence graph is used to test it.
5. Discuss the advantages, limitations, and practical applications of transactional processing.

Topic 04: Concurrency Control Techniques

Complete university-level notes in English | Definitions, detailed explanations, SQL examples, schedules, and exam preparation

1. Introduction to Concurrency Control

Definition

Concurrency control is a set of techniques used by a Database Management System (DBMS) to manage the simultaneous execution of multiple transactions while maintaining data consistency and transaction isolation.

In a multi-user database, many users may read or update the same data at the same time. Without proper control, their operations can interfere with each other and produce incorrect results.

Real-world example

Suppose a bank account contains Rs. 20,000. Two transactions run simultaneously:

- Transaction T1 withdraws Rs. 3,000.
- Transaction T2 withdraws Rs. 5,000.

The correct final balance should be:

\[20,000-3,000-5,000=12,000 \]

If both transactions read the original balance and overwrite each other's updates, the final balance may be incorrect.

Concurrency control helps prevent this type of problem while allowing multiple users to access the database efficiently.

Objectives of concurrency control

1. Maintain database consistency.
2. Prevent conflicting updates from producing incorrect results.
3. Preserve transaction isolation.
4. Allow multiple transactions to execute concurrently where safe.
5. Prevent or manage problems caused by simultaneous database access.
6. Improve database reliability and throughput.

2. Problems Caused by Concurrent Transactions

Before studying the control techniques, it is important to understand the problems they are designed to prevent.

2.1 Lost Update Problem

Definition: A lost update occurs when two transactions read the same data item and subsequently update it, causing one transaction's update to overwrite the other's.

Example: An account contains Rs. 10,000.

Step	Transaction T1	Transaction T2
1	Reads 10,000	
2		Reads 10,000
3	Deducts 2,000 → 8,000	
4		Deducts 3,000 → 7,000

5	Writes 8,000	
6		Writes 7,000

The final balance is Rs. 7,000, but the correct balance is Rs. 5,000.

The update made by T1 has been lost.

Prevention: Appropriate locking, serializable isolation, or other concurrency-control mechanisms can prevent this anomaly.

2.2 Dirty Read Problem

Definition: A dirty read occurs when one transaction reads data written by another transaction that has not yet committed.

Example:

1. T1 changes an account balance from Rs. 10,000 to Rs. 15,000.
2. T2 reads the new balance of Rs. 15,000.
3. T1 rolls back its changes.
4. T2 has used a value that was never committed.

This can lead to incorrect decisions in another transaction.

Prevention: Use an isolation level that prevents dirty reads, such as Read Committed or stronger.

2.3 Non-Repeatable Read

Definition: A non-repeatable read occurs when a transaction reads the same row twice and obtains different values because another transaction modifies and commits that row between the reads.

Example:

1. T1 reads a student's marks as 70.
2. T2 changes the marks to 85 and commits.
3. T1 reads the same row again and gets 85.

T1 sees different values during its execution.

Prevention: Repeatable Read or stronger isolation, depending on the DBMS and the transaction's requirements.

2.4 Phantom Read

Definition: A phantom read occurs when a transaction repeats a query that selects a set of rows and sees a different set because another transaction inserts or deletes rows matching the query condition.

Example:

T1 runs:

```
SELECT *
FROM Students
WHERE marks >= 80;
```

Initially, five students satisfy the condition. T2 inserts another qualifying student and commits. When T1 repeats the query, it may return six students.

The additional matching row is called a phantom.

Prevention: Serializable isolation or an appropriate predicate-locking or range-locking mechanism.

Summary of concurrency problems

Problem	Main cause
Lost update	One update overwrites another
Dirty read	Reading uncommitted data
Non-repeatable read	A previously read row changes
Phantom read	The set of matching rows changes

3. Lock-Based Concurrency Control

3.1 Definition of a Lock

A lock is a mechanism used by a DBMS to control access to a data item while a transaction is using or modifying it.

A transaction must obtain an appropriate lock before performing an operation that requires protected access to the data.

Locks help prevent conflicting transactions from interfering with each other.

Real-world example

Imagine a university clerk is updating a student's fee record. Another clerk attempts to modify the same record at the same time.

The database can place a lock on that record so the second clerk must wait until the conflicting operation is safe to perform.

3.2 Types of Locks

The two fundamental lock types are:

- Shared lock (S-lock)
- Exclusive lock (X-lock)

A. Shared Lock (S-lock)

Definition: A shared lock allows a transaction to read a data item while preventing conflicting exclusive access.

Multiple transactions can hold shared locks on the same item simultaneously.

Example:

T1 and T2 both want to read the balance of account 101.

- T1 obtains an S-lock.
- T2 obtains an S-lock.
- Both transactions can read the balance.
- Neither can obtain an incompatible X-lock until the shared locks are released.

Typical notation:

\[S_1(X) \]

This means transaction T1 holds a shared lock on data item X.

B. Exclusive Lock (X-lock)

Definition: An exclusive lock allows a transaction to access a data item for modification while preventing other transactions from obtaining conflicting locks on that item.

Example:

T1 wants to update a student's marks.

- T1 obtains an X-lock on the student's record.
- T1 changes the marks.
- Another transaction requesting a conflicting lock must wait.
- The lock is released according to the DBMS's locking protocol.

Typical notation:

\[X_1(X) \]

This means transaction T1 holds an exclusive lock on item X.

3.3 Lock Compatibility Table

Existing lock	Requested S-lock	Requested X-lock
Shared (S)	Compatible	Not compatible
Exclusive (X)	Not compatible	Not compatible

Important: Compatibility determines whether two transactions can hold their requested locks simultaneously on the same protected resource.

3.4 Locking Example

Suppose T1 updates account X while T2 wants to read it.

T1: LOCK-X(X)

T1: READ(X)

T1: WRITE(X)

T2: REQUEST-S(X) → WAIT

T1: COMMIT

T1: UNLOCK(X)

T2: LOCK-S(X)

T2: READ(X)

T2 waits until T1 releases the exclusive lock. It then reads the data according to the database's isolation and locking rules.

Advantages of locking

- Prevents many conflicting read and write operations.
- Helps maintain isolation.
- Supports correct concurrent transaction execution.
- Can prevent lost updates.

Disadvantages of locking

- Transactions may wait for locks.
- Deadlocks can occur.
- Excessive locking can reduce performance.
- Lock management consumes system resources.

4. Two-Phase Locking (2PL)

4.1 Definition

Two-Phase Locking (2PL) is a concurrency-control protocol in which each transaction has two phases:

1. Growing phase
2. Shrinking phase

During the growing phase, a transaction can acquire locks but cannot release any locks.

During the shrinking phase, it can release locks but cannot acquire new locks.

The key rule is that once a transaction releases its first lock, it cannot acquire any new locks under basic 2PL.

4.2 Growing Phase

In the growing phase, a transaction may:

- Acquire shared locks.
- Acquire exclusive locks.
- Upgrade a shared lock to an exclusive lock, when permitted by the protocol.

It cannot release locks during this phase.

4.3 Shrinking Phase

In the shrinking phase, a transaction may:

- Release shared locks.
- Release exclusive locks.
- Downgrade a lock where permitted.

It cannot acquire new locks after entering this phase.

Example of 2PL

Suppose transaction T1 transfers money between accounts A and B.

T1: LOCK-X(A) ← Growing phase

T1: LOCK-X(B) ← Growing phase

T1: READ(A)

T1: WRITE(A)

T1: READ(B)

T1: WRITE(B)

T1: UNLOCK(A) ← Shrinking phase

T1: UNLOCK(B)

T1: COMMIT

Once T1 releases A, it cannot acquire another lock under basic 2PL.

This example illustrates the two phases. In practical systems, the precise ordering of commit and lock release depends on the locking protocol being used.

4.4 Types of Two-Phase Locking

A. Basic 2PL

The transaction follows the growing and shrinking phases.

Benefit: Guarantees conflict serializability.

Limitation: Deadlocks may occur, and some schedules may allow cascading rollbacks.

B. Strict 2PL

A transaction holds its exclusive locks until it commits or aborts.

Benefits:

- Prevents other transactions from reading or overwriting uncommitted writes.
- Simplifies recovery.
- Prevents cascading aborts caused by reading uncommitted writes.

Strict 2PL is widely used in database systems.

C. Rigorous 2PL

A transaction holds both shared and exclusive locks until it commits or aborts.

Difference from strict 2PL: Strict 2PL requires exclusive locks to be retained, whereas rigorous 2PL retains both shared and exclusive locks.

D. Conservative (Static) 2PL

A transaction obtains all the locks it will need before it begins executing.

If all required locks cannot be granted, the transaction waits without acquiring any of them.

Benefit: Prevents deadlocks caused by transactions waiting for one another's locks.

Limitation: The transaction must know its locking requirements in advance, and acquiring all locks early can reduce concurrency.

4.5 Comparison of 2PL Variants

Protocol	Main rule	Deadlocks possible?
Basic 2PL	Growing then shrinking	Yes
Strict 2PL	Retain X-locks until commit/abort	Yes
Rigorous 2PL	Retain S- and X-locks until commit/abort	Yes
Conservative 2PL	Obtain all required locks before execution	No, for the protocol's lock-waiting mechanism

Limitations of 2PL

Although 2PL guarantees conflict serializability, it does not automatically prevent deadlocks. The DBMS still needs a deadlock-handling strategy, such as detection, prevention, or timeout-based recovery.

5. Timestamp-Based Concurrency Control

5.1 Definition

Timestamp-based concurrency control is a protocol that orders transactions using unique timestamps and ensures that conflicting operations follow the required timestamp order.

Each transaction receives a timestamp when it starts.

For example:

\[TS(T_1)=10 \]

\[TS(T_2)=20 \]

Since T1 has the smaller timestamp, T1 is considered older than T2.

The protocol attempts to make the execution equivalent to the timestamp order.

5.2 Read and Write Timestamps

For each data item X, the DBMS may maintain:

- \(RTS(X)\): The largest timestamp of a transaction that has successfully read X.
- \(WTS(X)\): The largest timestamp of a transaction that has successfully written X.

These values help the DBMS determine whether an operation violates timestamp ordering.

5.3 Read Rule

Suppose transaction T wants to read X.

If

\[TS(T)<WTS(X) \]

then a newer transaction has already written X. Allowing T to read it would violate the required timestamp order, so the basic timestamp-ordering protocol rejects the read and aborts T.

Otherwise, the read can proceed, and the DBMS updates:

\[RTS(X)=\max(RTS(X),TS(T)) \]

5.4 Write Rule

Suppose transaction T wants to write X.

Under the basic timestamp-ordering protocol:

- If \(TS(T)<RTS(X)\), a newer transaction has already read X. The write is rejected and T is aborted.

- If \(TS(T)<WTS(X)\), a newer transaction has already written X. The write is rejected and T is aborted.
- Otherwise, the write is allowed, and \(WTS(X)\) is updated to \(TS(T)\).

Some timestamp-based variants use different rules. For example, Thomas's Write Rule can ignore certain obsolete writes instead of aborting the transaction.

5.5 Worked Example

Assume:

\[TS(T_1)=10,\quad TS(T_2)=20 \]

Initially:

\[RTS(X)=0,\quad WTS(X)=0 \]

T2 writes X first:

\[WTS(X)=20 \]

Later, T1 tries to read X.

The read rule checks:

\[TS(T_1)<WTS(X) \]

\[10<20 \]

The condition is true. Therefore, under basic timestamp ordering, T1's read is rejected and T1 is aborted.

Advantages

- Provides a clear ordering of transactions.
- Does not require ordinary read/write locks.
- Avoids deadlocks caused by waiting for these locks.
- Can be effective for workloads with suitable access patterns.

Disadvantages

- Transactions may be aborted and restarted repeatedly.

- Older transactions can be disadvantaged.
- Timestamp metadata requires management.
- High conflict rates can reduce performance.

6. Deadlocks in DBMS

6.1 Definition

A deadlock occurs when two or more transactions wait indefinitely for resources held by one another, so none of them can proceed without intervention.

Deadlocks are common concerns in lock-based concurrency control.

6.2 Real-world example

Suppose two transactions need to update accounts A and B.

- T1 locks account A and requests account B.
- T2 locks account B and requests account A.

T1 cannot proceed until T2 releases B, but T2 cannot proceed until T1 releases A.

Neither transaction can continue.

6.3 Deadlock Schedule

T1: LOCK-X(A)

T2: LOCK-X(B)

T1: REQUEST-X(B) → WAIT

T2: REQUEST-X(A) → WAIT

T1 waits for T2, and T2 waits for T1.

This creates a deadlock.

6.4 Necessary Conditions for Deadlock

Four conditions are traditionally associated with deadlocks.

1. Mutual Exclusion

At least one resource can be held by only one transaction at a time.

Example: An exclusive lock on a particular data item.

2. Hold and Wait

A transaction holds one resource while waiting for another.

Example: T1 holds A while requesting B.

3. No Preemption

Resources cannot simply be taken away from a transaction; they must be released according to the system's rules.

4. Circular Wait

A circular chain of transactions exists, with each waiting for a resource held by the next.

Example:

\[T_1\rightarrow T_2\rightarrow T_1 \]

When all four conditions hold, a deadlock is possible.

6.5 Deadlock Detection

One common approach is to use a Wait-For Graph (WFG).

- Each node represents a transaction.
- An edge \(T_1\rightarrow T_2\) means T1 is waiting for a resource held by T2.
- A cycle indicates a deadlock in the standard single-instance resource model used for this graph.

Example:

T1

T2

T2 waits for T1: a cycle exists.

The DBMS can detect the cycle and select a transaction as the deadlock victim.

6.6 Deadlock Prevention

Deadlock prevention ensures that at least one of the four necessary conditions cannot arise, or uses a protocol that avoids circular waiting.

Common techniques include:

- Resource ordering: Require transactions to acquire resources in a fixed order.
- Conservative 2PL: Acquire all required locks before execution.
- Wait-Die: An older transaction may wait for a younger one, but a younger transaction requesting a resource held by an older one is aborted.
- Wound-Wait: An older transaction can force a younger lock holder to abort; a younger transaction waits for an older one.

Wait-Die and Wound-Wait use transaction timestamps to prevent cycles in the waiting relationships.

6.7 Deadlock Recovery

When a deadlock is detected, the DBMS can:

1. Select a victim transaction.
2. Roll back the victim, partially or completely as supported.
3. Release its locks.
4. Allow the remaining transaction to continue.
5. Restart the aborted transaction if appropriate.

Victim selection may consider transaction age, amount of work completed, locks held, and rollback cost.

6.8 Deadlock Prevention vs. Detection

Deadlock prevention	Deadlock detection

Attempts to stop deadlocks from forming	Allows deadlocks to occur
Uses ordering or timestamp policies	Searches for cycles in waiting relationships
May restrict concurrency	May permit greater concurrency
Can cause preventive aborts or waiting	Requires recovery after detection

7. Comparison of Concurrency Control Techniques

Technique	Basic mechanism	Main advantage	Main limitation
Lock-based control	Shared and exclusive locks	Directly controls conflicting access	Waiting and deadlocks
Two-Phase Locking	Growing and shrinking phases	Guarantees conflict serializability	Deadlocks remain possible
Timestamp ordering	Orders operations by timestamps	Avoids lock-based deadlocks	Can cause frequent aborts
Optimistic control	Validates transactions before committing	Useful when conflicts are rare	Validation failures require retries

Optimistic concurrency control is an additional technique worth knowing at BS level. It typically consists of read, validation, and write phases. Transactions execute with limited locking and are validated before their updates are committed.

8. Practical SQL Example: Preventing Lost Updates

Suppose a table stores account balances:

```
CREATE TABLE Accounts (
   account_id INT PRIMARY KEY,
   balance DECIMAL(12, 2) NOT NULL
);
```

A transaction can lock an account row before modifying it, where the DBMS supports this syntax:

```
START TRANSACTION;


SELECT balance

FROM Accounts

WHERE account_id = 101

FOR UPDATE;


UPDATE Accounts

SET balance = balance - 3000

WHERE account_id = 101

  AND balance >= 3000;


COMMIT;
```

Explanation

- The transaction starts.
- SELECT ... FOR UPDATE requests a lock on the selected row in supported systems.
- The update deducts Rs. 3,000 only when the balance is sufficient.
- COMMIT completes the transaction.

In production code, the application must also check whether the row exists and whether the UPDATE actually affected a row. SQL locking syntax and behavior vary between DBMS products, and a complete bank transfer should lock and update both accounts safely.

9. Advantages of Concurrency Control

1. Accuracy: Reduces incorrect results from conflicting updates.
2. Consistency: Helps preserve database constraints and correct transaction behavior.
3. Isolation: Controls interactions between concurrent transactions.
4. Data integrity: Protects shared records from unsafe modifications.
5. Performance: Enables safe parallel work rather than forcing all transactions to execute serially.
6. Reliability: Helps the DBMS manage conflicts systematically.

10. Limitations and Challenges

1. Lock contention: Transactions may wait for access to shared data.
2. Deadlocks: Some protocols allow transactions to wait cyclically.
3. Transaction aborts: Timestamp and optimistic protocols may require retries.
4. Overhead: Lock tables, timestamps, and validation consume resources.
5. Reduced concurrency: Stronger protection can limit parallel execution.
6. Implementation complexity: Correctness depends on choosing and configuring suitable protocols.

11. Important Exam Questions

Short questions

1. Define concurrency control.
2. What is the lost update problem?
3. Explain dirty reads and non-repeatable reads.
4. Define shared and exclusive locks.
5. What is lock compatibility?
6. Define Two-Phase Locking.
7. Differentiate strict 2PL and rigorous 2PL.

8. What is timestamp-based concurrency control?
9. Define deadlock.
10. What are the four necessary conditions for deadlock?
11. What is a Wait-For Graph?
12. Differentiate deadlock prevention and deadlock detection.

Long questions

1. Explain concurrency-control problems with appropriate transaction schedules.
2. Describe shared and exclusive locks with a compatibility table and examples.
3. Explain basic, strict, rigorous, and conservative Two-Phase Locking.
4. Explain timestamp-based concurrency control, including the read and write rules.
5. Define deadlock and explain its conditions, detection, prevention, and recovery.
6. Compare lock-based, timestamp-based, and optimistic concurrency-control techniques.

Topic 05: Database Recovery Techniques

Complete university-level notes in English | Definitions, examples, recovery logs, checkpoints, undo, redo, backup recovery, and exam preparation

1. Introduction to Database Recovery

Definition

Database recovery is the process of restoring a database to a correct, consistent, and usable state after a failure, such as a system crash, transaction error, power failure, or storage-device failure.

Database recovery is an essential function of a Database Management System (DBMS) because important information must not be permanently lost or left inconsistent when a failure occurs.

Real-world example

Suppose a bank transfers Rs. 5,000 from Ali's account to Ahmed's account.

Initially:

- Ali's balance = Rs. 20,000
- Ahmed's balance = Rs. 10,000

During the transaction:

1. Rs. 5,000 is deducted from Ali's account.
2. The database server crashes before Ahmed's account is credited.

The database may temporarily contain an incomplete transaction. Recovery mechanisms determine which changes must be undone and which committed changes must be redone.

After correct recovery, the database should reflect a valid state:

- Ali's balance = Rs. 20,000 and Ahmed's balance = Rs. 10,000 if the transfer did not commit.
- Ali's balance = Rs. 15,000 and Ahmed's balance = Rs. 15,000 if the transfer committed successfully.

The database must not permanently retain only half of the transfer.

2. Objectives of Database Recovery

The main objectives are:

1. Data preservation: Protect database information from permanent loss.
2. Atomicity: Prevent incomplete transactions from remaining applied.
3. Durability: Preserve committed changes after failures.
4. Consistency: Restore the database to a valid state.

5. Availability: Restore database services as quickly as practical.
6. Business continuity: Reduce interruptions to banking, education, healthcare, and other services.

3. Types of Database Failures

Understanding failure types helps us select the appropriate recovery technique.

3.1 Transaction Failure

A transaction failure occurs when an individual transaction cannot complete successfully.

Possible causes:

- Invalid input.
- Constraint violation.
- Insufficient account balance.
- Deadlock victim selection.
- Application error.
- Explicit rollback.

Example: A transaction attempts to withdraw Rs. 10,000 from an account containing Rs. 5,000 when overdrafts are prohibited. The transaction is rejected or rolled back.

Recovery approach: Undo the transaction's uncommitted changes, as necessary.

3.2 System Failure

A system failure occurs when the DBMS or operating system stops unexpectedly, while persistent storage may remain intact.

Possible causes:

- Power outage.
- Operating-system crash.
- Database server crash.

- Hardware restart.

Example: A university database server crashes while recording examination results.

Recovery approach: Use transaction logs and recovery procedures to undo incomplete transactions and restore committed changes when necessary.

3.3 Media Failure

A media failure occurs when the storage containing database files becomes damaged or inaccessible.

Possible causes:

- Hard-disk failure.
- SSD failure.
- Storage corruption.
- Physical damage to storage hardware.

Example: A server's disk containing student records becomes unreadable.

Recovery approach: Restore a backup and apply available archived logs or transaction logs to recover changes made after the backup.

3.4 Communication Failure

Communication failures are particularly important in distributed databases.

Possible causes:

- Network interruption.
- Connection loss between database servers.
- Failure during a distributed transaction.

Example: An online payment service loses its connection to the bank after sending a payment request.

The application may not know whether the payment committed. It should verify the transaction's status rather than blindly repeat the payment.

3.5 Human Error

Human errors can also damage database information.

Examples:

- Accidental deletion of records.
- Incorrect bulk updates.
- Dropping a table.
- Changing the wrong database.

Recovery may require point-in-time recovery, backups, transaction logs, or restoration of an earlier copy of the affected data.

4. Log-Based Recovery

4.1 Definition

Log-based recovery is a database recovery technique in which the DBMS records information about transaction operations in a log so that changes can be undone or redone after a failure.

A log is a chronological record of transaction events, such as transaction starts, writes, commits, and aborts.

Logs provide the information needed to determine what happened before a failure.

4.2 Example of a Transaction Log

Suppose transaction T1 changes account A from Rs. 10,000 to Rs. 8,000.

A simplified log might contain:

<START T1>

<T1, A, 10000, 8000>

<COMMIT T1>

Explanation:

- <START T1> indicates that transaction T1 has started.
- <T1, A, 10000, 8000> records the old and new values of A.

- <COMMIT T1> indicates that T1 has committed.

The old value is useful for undo, and the new value is useful for redo.

Actual DBMS logs can be more detailed and may use physical, logical, or physiological log records.

4.3 Write-Ahead Logging (WAL)

Definition

Write-Ahead Logging (WAL) is a recovery principle requiring the relevant log records to be made durable before the corresponding modified database pages are written to persistent storage.

A crucial rule is:

The log must be safely written before the database page containing the corresponding change is written.

For transaction durability, the commit record must also be made durable before the DBMS confirms a successful commit, according to the system's configured guarantees.

Why is WAL important?

Suppose T1 changes account A from Rs. 10,000 to Rs. 8,000.

If the database page is written to disk but the log containing the old and new values is not safely stored, a later crash may leave the DBMS without sufficient information to recover correctly.

WAL protects the recovery information by ensuring that the required log records reach durable storage first.

Typical WAL sequence

1. Transaction modifies data

The change is made in memory.

2. Log records are written

Required recovery information is recorded.

3. Log is made durable

The log is safely persisted before the corresponding data page.

4. Database page is written later

The DBMS can use the durable log during recovery.

The database page does not necessarily have to be written before the transaction commits. WAL permits the DBMS to manage page writes independently while preserving recovery guarantees.

4.4 Types of Log Records

Log record	Purpose
Start record	Identifies the beginning of a transaction
Update record	Records a data modification
Commit record	Identifies successful transaction completion
Abort record	Records transaction abortion
Checkpoint record	Identifies a recovery checkpoint
Compensation log record	Records an action performed during undo in ARIES-style recovery

A compensation log record helps recovery resume safely if another crash occurs while recovery is in progress.

5. Deferred and Immediate Database Modification

Two traditional recovery approaches differ in when modified data is written and which log information is required.

5.1 Deferred Database Modification

Definition

Deferred database modification delays applying a transaction's updates to the database until the transaction has committed.

The log records enough information to redo committed updates if necessary.

This is commonly described as a redo-only approach in its simplified form.

Example

Transaction T1 changes X from 100 to 150.

<START T1>

<T1, X, 150>

<COMMIT T1>

The simplified log records the new value.

If T1 commits but its update has not yet been reflected in the database file when the system crashes, recovery can redo the committed update.

Advantages

- Simplifies recovery in the traditional deferred-update model.
- Uncommitted updates are not applied to the database under the model's assumptions.

Limitations

- The DBMS must manage delayed writes.
- It may not suit every storage and performance design.

5.2 Immediate Database Modification

Definition

Immediate database modification permits database pages to be written before a transaction commits.

Therefore, a page may contain changes made by an uncommitted transaction.

The log must contain enough information to undo uncommitted changes and, where necessary, redo committed changes.

Example

Transaction T1 changes X from 100 to 150.

<START T1>

<T1, X, 100, 150>

The page containing X may be written before T1 commits.

If the system crashes and T1 has not committed, recovery can use the old value of 100 to undo the change.

Comparison

Deferred modification	Immediate modification
Updates are deferred until commit in the traditional model	Pages may be written before commit
Simplified model uses redo	May require both undo and redo
Uncommitted changes are not applied under the model's assumptions	Uncommitted changes may reach disk

Modern DBMS implementations often use more sophisticated variants, so the precise recovery algorithm depends on the system.

6. Undo and Redo Operations

Undo and redo are two fundamental operations in database recovery.

6.1 Undo

Definition

Undo reverses a transaction's effects by restoring data to its earlier state.

It is used when changes from a transaction must not remain applied, typically because the transaction did not commit successfully.

Example

Before the transaction:

\[X=100 \]

Transaction T1 changes X to 150, but then aborts.

Undo restores the previous value:

$$ X=100 $$

A simplified log record contains:

<T1, X, 100, 150>

During undo, the DBMS uses the old value, 100.

When is undo needed?

- Transaction rollback.
- Recovery of incomplete transactions after a crash.
- Reversing uncommitted changes during recovery.

6.2 Redo

Definition

Redo reapplies a transaction's change so that the database reflects a successfully committed operation.

Redo is useful when a transaction committed, but its updates were not all written to the database files before a crash.

Example

Before the transaction:

$$ X=100 $$

T1 changes X to 150 and commits. However, the database page containing X has not yet been written to persistent storage.

After a crash, the log indicates that T1 committed. Recovery reapplies the update:

$$ X=150 $$

When is redo needed?

- Restoring committed changes after a crash.

- Reapplying updates missing from database pages.
- Recovering changes recorded in transaction logs after restoring a backup.

6.3 Difference Between Undo and Redo

Undo	Redo
Reverses changes	Reapplies changes
Uses previous values or inverse operations	Uses logged updates or new values
Commonly applies to incomplete transactions	Commonly applies to committed changes needing restoration
Restores the appropriate earlier state	Restores the committed result

Easy way to remember:

- Undo = Take the change back.
- Redo = Apply the change again.

Combined example

Suppose the log contains:

<T1, A, 10000, 8000>

<COMMIT T1>

<T2, B, 5000, 3000>

The system crashes after T2's update record but before T2 commits.

Recovery determines:

- T1 committed, so its change may need to be redone.
- T2 did not commit, so its change must be undone if it was applied.

Final values:

- A = Rs. 8,000.
- B = Rs. 5,000.

This is a simplified illustration; real recovery algorithms also track log sequence numbers, page states, and recovery progress.

7. Checkpointing

7.1 Definition

A checkpoint is a recovery mechanism that records a known recovery position in the database log and may coordinate the recording of relevant transaction and database state information.

Checkpoints reduce the amount of log history that must be examined during recovery, although the exact amount depends on the recovery algorithm.

Why are checkpoints necessary?

Without checkpoints, a DBMS might have to examine a large portion of its transaction log after every crash.

For example, a database may have accumulated several gigabytes of log records since it was first started. A checkpoint can help recovery focus on relevant activity rather than always starting from the beginning of the log.

7.2 How Checkpointing Works

A simplified checkpoint process is:

1. The DBMS records a checkpoint in the log.
2. It records relevant transaction or recovery-state information.
3. Depending on the implementation, it may flush certain log records or database pages.
4. Normal transaction processing continues.
5. If a crash occurs, recovery uses the checkpoint information to determine where and how to begin examining the log.

Modern systems may use fuzzy checkpoints, which do not require all transactions to stop or every dirty page to be written at the same time.

7.3 Example of a Checkpoint

Consider this simplified log:

<START T1>

<T1, A, 100, 150>

<COMMIT T1>

<CHECKPOINT>

<START T2>

<T2, B, 200, 250>

Suppose the system crashes after T2 modifies B but before it commits.

Recovery can use the checkpoint and associated metadata to identify relevant transactions and log records.

- T1 committed before the checkpoint.
- T2 was active at the time of failure and had not committed.

The recovery procedure may not need to process all historical records from the beginning of the database's lifetime. However, it must still ensure that all required committed changes are present and all incomplete changes are appropriately handled.

7.4 Types of Checkpoints

A. Quiescent checkpoint

A quiescent checkpoint is taken when the DBMS temporarily reaches a state with no active transactions or otherwise coordinates a quiet point for checkpointing.

This approach is simple conceptually but may interrupt transaction processing.

B. Fuzzy checkpoint

A fuzzy checkpoint allows transactions to continue while checkpoint information is recorded.

It is commonly used in modern database systems because it reduces disruption to normal operations.

Advantages of checkpointing

- Can reduce crash-recovery time.
- Helps identify relevant log records.
- Improves recovery efficiency.
- Supports management of database log history.

Limitations

- Checkpoint processing consumes resources.
- Frequent checkpoints can create overhead.
- Infrequent checkpoints may increase recovery work.
- A checkpoint alone does not replace transaction logs or backups.

8. Database Recovery Using Logs

A DBMS uses transaction logs to determine the correct actions after a crash.

In a simplified undo/redo recovery model, transactions are classified according to whether they committed before the failure.

- Committed transactions: Redo their changes if required.
- Uncommitted transactions: Undo their changes if they were applied.

Example

Consider the following log:

<START T1>

<T1, A, 100, 150>

<COMMIT T1>

<START T2>

<T2, B, 200, 250>

<CRASH>

Assume the system uses a recovery method that requires undo and redo.

Transaction	Status at crash	Recovery action
T1	Committed	Redo if its update is missing
T2	Uncommitted	Undo if its update was applied

After recovery:

- A should be 150.
- B should be 200.

The exact recovery process depends on the log format, the database's page-writing strategy, and its recovery algorithm.

9. Backup and Recovery

9.1 Definition of Backup

A database backup is a separate copy of database data maintained so that the database can be restored after data loss, corruption, or other failures.

Backups are essential for recovering from failures that transaction logs alone cannot solve, such as a destroyed storage device.

9.2 Types of Database Backups

A. Full backup

A full backup copies the complete database or the complete scope specified by the backup operation.

Example: A university backs up its entire student database every Sunday.

Advantages:

- Straightforward restoration.
- Provides a complete backup baseline.

Disadvantages:

- Requires more storage.
- Can take longer to create.

B. Incremental backup

An incremental backup copies changes made since the previous relevant backup, commonly the previous incremental backup.

Example:

- Sunday: Full backup.
- Monday: Incremental backup of Monday's changes.
- Tuesday: Incremental backup of Tuesday's changes.

To restore Tuesday's state, the DBMS may need the full backup and the incremental backups in sequence.

Advantages: Smaller backups and potentially faster backup operations.

Disadvantages: Restoration can be more complex and may require several backup sets.

C. Differential backup

A differential backup copies changes made since the last full backup.

Example:

- Sunday: Full backup.
- Monday: Differential backup containing changes since Sunday.

- Tuesday: Differential backup containing changes since Sunday, including Monday's changes.

To restore Tuesday's state, the typical sequence is the full backup followed by Tuesday's differential backup.

Advantages: Restoration often requires fewer backup sets than an incremental strategy.

Disadvantages: Differential backups can grow larger over time until the next full backup.

Comparison

Feature	Full backup	Incremental backup	Differential backup
Copies	Entire backup scope	Changes since previous backup	Changes since last full backup
Backup size	Usually largest	Usually smaller	Grows between full backups
Restore complexity	Relatively simple	Often highest	Often moderate
Typical restore inputs	Full backup	Full plus required incrementals	Full plus latest differential

Actual behavior depends on the backup software and database platform.

9.3 Point-in-Time Recovery (PITR)

Definition

Point-in-time recovery restores a database to a particular time or recovery position using a suitable backup and the transaction logs available after that backup.

Example

A staff member accidentally deletes student records at 2:00 PM.

The organization has:

- A valid database backup from midnight.
- Transaction logs covering the period after midnight.

If the database system supports point-in-time recovery and the logs are complete, the administrator may restore the backup and replay changes up to just before the accidental deletion.

This can recover the database to a state before the error occurred.

Why PITR is useful

- Recovers from accidental deletion.
- Helps recover from incorrect bulk updates.
- Supports recovery to a chosen point before logical corruption.
- Reduces data loss when backups and logs are maintained properly.

9.4 Backup Best Practices

1. Schedule regular backups.
2. Keep backup copies separate from the primary database.
3. Maintain appropriate off-site or immutable copies.
4. Protect backups through encryption and access control.
5. Test restoration regularly.
6. Monitor backup completion and integrity.
7. Retain the transaction logs required for the desired recovery window.
8. Define Recovery Point Objective (RPO) and Recovery Time Objective (RTO).

RPO describes the maximum acceptable amount of data loss measured in time.

RTO describes the target time for restoring a service after disruption.

For example, an RPO of 15 minutes means the organization aims to limit data loss to no more than approximately 15 minutes of changes under its recovery plan. An RTO of one hour means the service should be restored within the one-hour target.

10. Crash Recovery Using ARIES

At BS 7th Semester level, it is useful to understand ARIES, a widely studied database recovery algorithm.

ARIES stands for Algorithms for Recovery and Isolation Exploiting Semantics.

It is designed for recovery in systems that use write-ahead logging and may write modified database pages before the corresponding transactions commit.

ARIES is commonly described through three phases.

10.1 Analysis Phase

The analysis phase examines the log, starting from an appropriate checkpoint, to determine the database state at the time of the crash.

It identifies information such as:

- Transactions that were active.
- Transactions that committed.
- Dirty pages that might need recovery.

10.2 Redo Phase

The redo phase repeats relevant logged actions to reconstruct the database state that existed immediately before the crash.

This is often described as repeating history.

The algorithm determines which changes need to be reapplied based on the log and page information.

10.3 Undo Phase

The undo phase reverses the effects of transactions that were incomplete at the time of the crash.

It uses log information to roll back their effects while recording compensation information as required.

ARIES summary

1. Analysis

Determine transaction and page state.

2. Redo

Repeat required logged actions.

3. Undo

Reverse incomplete transactions.

Why is ARIES important?

- Supports sophisticated crash recovery.
- Works with write-ahead logging.
- Uses checkpoints to limit recovery analysis.
- Supports recovery even when modified pages have been written before transactions commit.

The precise implementation of ARIES depends on the DBMS and its storage architecture.

11. Recovery Example: Complete Scenario

Suppose a university database contains the following records:

- Student A's fee balance = Rs. 10,000.
- Student B's fee balance = Rs. 20,000.

Two transactions execute:

- T1 records a payment of Rs. 2,000 from Student A.
- T2 records a payment of Rs. 3,000 from Student B.

The log contains:

<START T1>

<T1, A, 10000, 8000>

<COMMIT T1>

<START T2>

<T2, B, 20000, 17000>

<CHECKPOINT>

<CRASH>

Assume the checkpoint and log information are sufficient to identify the transaction states.

Recovery analysis

Transaction T1

- T1 has a commit record.
- Its change is committed.
- If the change is missing from the database page, recovery must restore it.

Transaction T2

- T2 has no commit record before the crash.
- Its change is incomplete.
- If the change was applied to a database page, recovery must undo it.

Expected final state

Student	Initial value	Final value after recovery
A	Rs. 10,000	Rs. 8,000
B	Rs. 20,000	Rs. 20,000

The example demonstrates the relationship between logs, checkpoints, undo, and redo.

12. Important Exam Questions

Short questions

1. Define database recovery.
2. What is a transaction log?
3. Define Write-Ahead Logging (WAL).
4. What is the purpose of a checkpoint?
5. Define undo and redo operations.
6. Differentiate deferred and immediate database modification.
7. What is a full backup?
8. Differentiate incremental and differential backups.
9. Define point-in-time recovery.
10. What are RPO and RTO?
11. What is ARIES?
12. Name the three phases of ARIES recovery.

Long questions

1. Explain database recovery and the different types of database failures.
2. Describe log-based recovery and explain Write-Ahead Logging with examples.
3. Explain undo and redo operations with transaction logs.
4. Discuss checkpointing, its types, advantages, and limitations.
5. Compare full, incremental, and differential backups.
6. Explain point-in-time recovery with a practical example.
7. Describe the three phases of the ARIES recovery algorithm.
8. Explain how a DBMS recovers committed and uncommitted transactions after a crash.

Topic 06: Query Processing and Optimization

Database Management System (DBMS) — BS Computer Science, 7th Semester

1. Introduction to Query Processing

Definition: Query processing is the process through which a Database Management System (DBMS) interprets an SQL query, determines an efficient way to execute it, and returns the required results.

When a user writes an SQL query, the DBMS does not simply execute it directly. It first checks the query, considers different execution methods, and selects an appropriate plan.

Example:

SELECT Name

FROM Student

WHERE Marks > 80;

This query retrieves the names of students whose marks are greater than 80.

The DBMS must determine how to find those students efficiently, especially if the Student table contains millions of records.

2. Stages of Query Processing

Query processing is generally divided into three main stages.

Stage 1: Parsing and Translation

Checks SQL syntax and converts the query into an internal representation.

Stage 2: Query Optimization

Evaluates possible execution plans and chooses an efficient plan.

Stage 3: Query Evaluation

Executes the selected plan and produces the result.

Stage 1: Parsing and Translation

The DBMS checks whether the SQL query is valid.

It checks:

- Syntax: Is the SQL statement written correctly?
- Semantic validity: Do the table and column names exist? Are the operations valid?
- Authorization: Is the user permitted to access the requested data?
- Translation: The query is converted into an internal form, often involving relational algebra or a query tree.

Example of an incorrect query:

SELEC Name FROM Student;

The keyword SELECT is misspelled, so the DBMS reports a syntax error.

Stage 2: Query Optimization

Query optimization is the process of choosing an efficient execution plan from several possible plans that produce the same result.

Suppose a table contains one million student records. The DBMS may:

1. Read every record.
2. Use an index on the Marks column.
3. Apply a filter before processing other operations.

The optimizer estimates the costs of available alternatives and selects an appropriate plan.

Stage 3: Query Evaluation (Execution)

The execution engine runs the chosen query plan.

It may read records from storage, use indexes, filter rows, join tables, sort results, and return the final output.

Important: The best execution method depends on the data, indexes, statistics, database configuration, and query conditions.

3. Query Execution Plan

Definition: A query execution plan is a sequence of operations selected by the DBMS to execute an SQL query.

Consider:

SELECT Name

FROM Student

WHERE Marks > 80;

A possible logical execution plan is:

Projection

Return only Name

Selection

Keep rows where Marks > 80

Student Table

Source data

The diagram shows the logical operations. A physical execution plan specifies how these operations are implemented, such as using an index scan or a full table scan.

Logical vs. Physical Query Plan

Logical query plan	Physical query plan
Describes what operations are required.	Describes how operations are performed.
Uses operations such as selection, projection and join.	Uses methods such as index scan, hash join and sorting.
Focuses on the logical structure.	Focuses on actual execution and cost.

Example: A logical plan may specify a selection on Marks > 80. A physical plan may perform that selection through an index range scan on the Marks index.

4. Query Processing Operations

4.1 Selection

Definition: Selection retrieves rows that satisfy a specified condition.

In relational algebra, selection is represented by \(\sigma\) (sigma).

SQL example:

```
SELECT *
FROM Student
WHERE Marks > 80;
```

This returns only students whose marks are greater than 80.

4.2 Projection

Definition: Projection retrieves only the required columns from a relation.

In relational algebra, projection is represented by \(\pi\) (pi).

```
SELECT Name, Marks
FROM Student;
```

This returns only the Name and Marks columns.

Optimization benefit: Reading fewer columns can reduce data processing and memory usage.

4.3 Sorting

Sorting arranges records according to a specified order.

```
SELECT Name, Marks
FROM Student
ORDER BY Marks DESC;
```

This displays students in descending order of marks.

Sorting large datasets may require significant memory and disk operations.

4.4 Aggregation

Aggregation calculates summary values from records.

Common aggregate functions include:

- COUNT() — counts rows or values.
- SUM() — calculates a total.
- AVG() — calculates an average.
- MIN() — finds the minimum.
- MAX() — finds the maximum.

Example:

SELECT AVG(Marks)

FROM Student;

This calculates the average marks of the students included in the query.

4.5 Join

Definition: A join combines related rows from two or more tables.

Suppose there are two tables:

Student

StudentID	Name
1	Ali
2	Sara

Result

StudentID	Marks
1	85
2	92

SQL example:

SELECT Student.Name, Result.Marks

FROM Student

JOIN Result

ON Student.StudentID = Result.StudentID;

The DBMS matches rows using StudentID and returns the student's name along with their marks.

Joins can be expensive when tables are large, so join optimization is important.

5. Database Access Methods

An access method determines how the DBMS retrieves records from a table.

5.1 Full Table Scan

Definition: A full table scan reads the table's records to find the required data.

Example:

SELECT *

FROM Student

WHERE Marks = 85;

If there is no useful index, the DBMS may inspect every row.

Advantages

- Simple and effective for small tables.
- Can be efficient when a large proportion of rows is needed.
- Does not require an index.

Disadvantages

- Can be slow for large tables when only a few rows match.
- May require substantial disk I/O.

5.2 Index Scan

Definition: An index scan uses an index to locate relevant records instead of necessarily reading the entire table.

Example:

```
CREATE INDEX idx_student_marks
ON Student(Marks);
```

The DBMS may use this index for:

```
SELECT Name
FROM Student
WHERE Marks = 85;
```

An index can speed up searches, but it consumes storage and adds maintenance overhead when data changes.

Important: Creating an index does not guarantee that the DBMS will use it. The optimizer chooses the access method based on estimated cost.

6. Types of Database Indexes

6.1 B+ Tree Index

A B+ tree is a balanced tree structure commonly used in database indexes.

It supports efficient searches, insertions, deletions and range queries.

Example:

```
SELECT *
FROM Student
WHERE Marks BETWEEN 70 AND 90;
```

A B+ tree index on Marks can efficiently locate values within this range.

Best suited for: Equality searches, range searches and ordered retrieval.

6.2 Hash Index

A hash index uses a hash function to locate entries.

Example:

```
SELECT *
FROM Student
```

WHERE StudentID = 101;

A hash index may be efficient for equality searches.

Best suited for: Exact-match queries, depending on the DBMS.

Limitation: Hash indexes generally do not support range searches as efficiently as B+ tree indexes.

6.3 Clustered Index

A clustered index determines how table rows are organized in the storage structure of a database system.

It can improve access to related rows, particularly for range queries.

The exact implementation differs between DBMS products. For example, SQL Server uses the clustered index to define the table's row organization, while other systems have different storage models.

6.4 Non-Clustered Index

A non-clustered index is a separate index structure that stores indexed values and references to the corresponding table rows.

It can speed up searches without determining the table's overall row organization.

Example: A student table may be organized using StudentID, while a separate non-clustered index supports searches by Marks.

7. Query Optimization

Definition: Query optimization is the process of selecting an efficient way to execute a database query while preserving its intended result.

The main objectives are to reduce:

- Disk input/output (I/O).
- CPU processing time.
- Memory consumption.
- Intermediate result sizes.

- Overall query response time.

There are two major approaches.

7.1 Heuristic-Based Optimization

Heuristic optimization applies general rules to improve a query plan.

Common rules include:

1. Apply selections early to reduce the number of rows.
2. Apply projections early to reduce unnecessary columns.
3. Avoid unnecessary sorting.
4. Perform joins in a suitable order.
5. Avoid creating unnecessarily large intermediate results.

For example, filtering students before joining their records to other tables can reduce the number of rows involved in the join.

These rules are useful, but a heuristic does not always produce the lowest-cost plan.

7.2 Cost-Based Optimization

Cost-based optimization estimates the cost of alternative execution plans and selects a plan expected to be efficient.

It may consider:

- Number of rows in each table.
- Available indexes.
- Number of disk pages.
- Data distribution.
- Number of matching rows.
- CPU and I/O costs.
- Available memory.

- Join algorithms and join order.

Example: If a condition matches only 10 rows out of one million, an index scan may be cheaper than a full table scan. If it matches 900,000 rows, a full table scan may be more efficient.

8. Important Query Optimization Techniques

8.1 Predicate Pushdown

Definition: Predicate pushdown means applying a filter as early as possible in the execution plan.

Example:

SELECT S.Name, R.Marks

FROM Student S

JOIN Result R

ON S.StudentID = R.StudentID

WHERE R.Marks > 80;

The optimizer may filter Result rows with marks greater than 80 before joining them with Student.

Benefit: Fewer rows need to participate in the join.

8.2 Projection Pushdown

Definition: Projection pushdown means retrieving only the columns needed by later operations as early as possible.

If a query needs only StudentID and Name, processing unrelated columns may be unnecessary.

Benefit: Reduces the size of intermediate data and may reduce I/O and memory usage.

8.3 Join Ordering

When a query joins three or more tables, the order of joins can affect performance.

Suppose:

- Student contains 100,000 rows.
- Enrollment contains 500,000 rows.
- Department contains 20 rows.

The optimizer considers which join order is likely to produce smaller intermediate results. It may be beneficial to apply selective filters before joining large tables.

Important: The smallest table is not always the best table to join first. The choice depends on join conditions, data distribution, indexes and estimated costs.

8.4 Index Selection

The optimizer decides whether an available index is useful for a particular query.

Indexes are especially helpful for selective conditions, joins and ordering operations when they match the query requirements.

However, too many indexes can slow down INSERT, UPDATE and DELETE operations because indexes must also be maintained.

8.5 Avoiding Unnecessary Operations

The DBMS may improve a query by removing redundant operations or avoiding unnecessary sorting, duplicate elimination or repeated data access.

For example, do not use SELECT * when only two columns are required:

SELECT Name, Marks

FROM Student;

This makes the requested data explicit and may reduce the amount of data transferred and processed.

9. Join Algorithms

A DBMS can use different algorithms to join tables. The best choice depends on table sizes, indexes, available memory and data ordering.

9.1 Nested-Loop Join

Definition: A nested-loop join compares rows from one table with rows from another table.

Conceptually:

1. Select one row from the first table.
2. Check matching rows in the second table.
3. Repeat for the remaining rows.

Advantages

- Simple.
- Can work well when one table is small.
- An index on the inner table can make it efficient.

Disadvantages

- Can be expensive when both tables are large and no useful index is available.

9.2 Index Nested-Loop Join

This method uses an index on the inner table to find matching rows.

For every relevant row from the outer table, the DBMS searches the index for matching inner-table rows.

Best suited for: Situations where the outer input is relatively small and a useful index exists on the join column of the inner table.

9.3 Sort-Merge Join

Definition: A sort-merge join sorts the inputs on the join key, if necessary, and then scans them to find matching values.

Advantages

- Useful for large inputs.
- Can be efficient when inputs are already sorted.
- Can support certain ordered processing requirements.

Disadvantages

- Sorting unsorted inputs may be expensive.

9.4 Hash Join

Definition: A hash join uses a hash table to match rows based on join keys.

The general process is:

1. Build a hash table from one input.
2. Read the other input and probe the hash table for matches.
3. Return matching rows.

Advantages

- Often efficient for large equi-joins.
- Can avoid sorting.

Disadvantages

- Requires memory for the hash table.
- If memory is insufficient, additional disk operations may be needed.
- It is not generally suitable for arbitrary range-join conditions.

Comparison of Join Algorithms

Algorithm	Main idea	Common use
Nested-loop join	Compare rows between inputs	Small inputs or indexed lookups
Index nested-loop join	Search inner input through an index	Selective joins with a useful index
Sort-merge join	Sort and merge on join keys	Large inputs, especially sorted data
Hash join	Build and probe a hash table	Large equality joins

10. Query Cost Estimation

Definition: Query cost estimation predicts the resources required to execute a query plan.

A simplified cost model can be expressed as:

\[\text{Total Cost} \approx \text{I/O Cost}+\text{CPU Cost} \]

Real DBMS cost models may also account for memory, parallel execution, network transfer and other factors.

10.1 Disk I/O Cost

Disk I/O refers to reading or writing data pages.

Because database tables and indexes may be stored on disk, the number of pages accessed can strongly affect performance.

For example, a full table scan may read many pages, while an index scan may access fewer pages when only a small number of records match.

10.2 CPU Cost

CPU cost includes comparisons, calculations, expression evaluation, hashing, sorting and other processing.

A query that performs many comparisons or complex calculations may consume significant CPU resources even if its disk I/O is limited.

10.3 Cardinality Estimation

Definition: Cardinality estimation predicts the number of rows produced by an operation.

Suppose a table contains 100,000 records and the optimizer estimates that 5% satisfy a condition.

\[100{,}000 \times 0.05 = 5{,}000 \]

The estimated result contains 5,000 rows.

This estimate helps the optimizer compare different access methods and join plans.

10.4 Selectivity

Definition: Selectivity describes the proportion of rows that satisfy a condition.

$$\text{Selectivity}= \frac{\text{Matching Rows}}{\text{Total Rows}}$$

Example:

- Total rows: 10,000
- Matching rows: 100

$$\text{Selectivity}=\frac{100}{10,000}=0.01$$

The selectivity is 1%.

A highly selective condition matches a small proportion of rows. An index may be useful in such cases, although the optimizer must consider the actual index and storage costs.

10.5 Database Statistics

Database statistics provide information used by the optimizer, such as:

- Table row counts.
- Number of pages.
- Number of distinct values.
- Value distributions and histograms.
- Index characteristics.

If statistics are outdated or inaccurate, the optimizer may estimate costs poorly and select an inefficient plan.

11. Using EXPLAIN to Analyze Queries

Many database systems provide an EXPLAIN command or an equivalent tool for displaying a query execution plan.

Example in PostgreSQL:

```
EXPLAIN

SELECT Name
```

```
FROM Student
WHERE Marks > 80;
```

The output may show:

- Whether the DBMS uses a sequential scan or index scan.
- Estimated number of rows.
- Estimated cost.
- Operations required to execute the query.

PostgreSQL also supports:

```
EXPLAIN ANALYZE
SELECT Name
FROM Student
WHERE Marks > 80;
```

EXPLAIN ANALYZE executes the query and reports actual execution information along with the plan.

Caution: EXPLAIN ANALYZE really runs the statement. For queries that modify data, execution can change database contents unless the operation is appropriately controlled. Syntax and output vary by DBMS.

How to Read a Query Plan

When examining a plan, check:

1. Scan type: Is it a sequential scan, index scan or another method?
2. Estimated rows: How many rows does the optimizer expect?
3. Actual rows: When actual execution data is available, how many rows were processed?
4. Join method: Is a nested-loop, hash or merge join being used?
5. Cost and timing: Which operations consume the most estimated resources or actual time?

6. Estimation errors: Are actual row counts substantially different from estimated counts?

Large differences between estimated and actual row counts can indicate stale statistics, unexpected data distribution or complex conditions that are difficult to estimate.

12. Complete Practical Example

Suppose a university database contains two tables.

Student

- StudentID
- Name
- DepartmentID

Result

- StudentID
- Marks

The requirement is to display the names of students who scored more than 80.

SQL Query

```
SELECT S.Name
FROM Student S
JOIN Result R
ON S.StudentID = R.StudentID
WHERE R.Marks > 80;
```

Step 1: Parsing

The DBMS checks the SQL syntax, table names, column names and permissions.

Step 2: Translation

The DBMS represents the query as logical operations:

- Filter results where Marks > 80.
- Join qualifying results with students.
- Return only student names.

Step 3: Optimization

The optimizer considers alternatives, such as:

- Scanning the entire Result table.
- Using an index on Result.Marks, if available.
- Choosing a join algorithm.
- Choosing a join order.

It estimates the costs using table statistics and available indexes.

Step 4: Execution

The execution engine runs the chosen plan and returns the names of students whose marks exceed 80.

Possible Performance Improvements

CREATE INDEX idx_result_marks

ON Result(Marks);

An index on Result.Marks may help when the condition is selective.

An index on Result.StudentID may also help certain join plans. Whether either index improves performance depends on the data and the DBMS.

Important: Always examine the execution plan and test with representative data before deciding that an index or rewrite improves performance.

13. Advantages of Query Optimization

1. Faster query execution: Reduces unnecessary processing.
2. Reduced disk I/O: Can minimize pages read from storage.
3. Better resource utilization: Uses CPU and memory more efficiently.

4. Improved scalability: Helps queries remain practical as datasets grow.
5. Better response time: Improves the experience of database users and applications.
6. Efficient joins: Chooses suitable join algorithms and orders.
7. Lower operating costs: Efficient queries may reduce resource consumption.

14. Challenges of Query Optimization

1. Large search space: Many possible execution plans may exist.
2. Inaccurate statistics: Incorrect estimates can produce poor choices.
3. Changing data: A plan that performs well today may become less effective as data grows.
4. Index overhead: Indexes consume storage and increase the cost of data modifications.
5. Memory limitations: Hash joins and sorting may require more memory than is available.
6. Complex queries: Nested queries, multiple joins and complex predicates make optimization difficult.
7. Different DBMS implementations: Available algorithms and optimization behavior vary by database system.

15. Important Exam Questions

Short Questions

1. Define query processing.
2. What is query optimization?
3. Name the three main stages of query processing.
4. Differentiate between a logical and a physical query plan.
5. What is a full table scan?
6. Define an index and state two advantages.

7. What is cardinality estimation?
8. Define selectivity.
9. What is predicate pushdown?
10. What is the purpose of the EXPLAIN command?

Long Questions

1. Explain the complete process of query processing with a suitable diagram.
2. Discuss heuristic-based and cost-based query optimization.
3. Explain B+ tree indexes, hash indexes, clustered indexes and non-clustered indexes.
4. Describe nested-loop, index nested-loop, sort-merge and hash join algorithms.
5. Explain query cost estimation, including I/O cost, CPU cost, cardinality and selectivity.
6. Discuss the major query optimization techniques with SQL examples.
7. Explain how to analyze a query execution plan and improve query performance.

Topic 07: Database Programming

Database Management System (DBMS) — BS Computer Science, 7th Semester

1. Introduction to Database Programming

Definition: Database programming is the process of writing programs and commands that interact with a database to store, retrieve, update, delete, validate, and process data.

Database programming combines SQL with programming features such as variables, conditions, loops, functions, procedures, and exception handling.

Example

Suppose a university has a database containing student records. Database programming can be used to:

- Add a new student.
- Update student marks.
- Calculate the average marks.
- Generate a result report.
- Automatically update records when certain events occur.
- Connect a website or application to the database.

Common database programming technologies include:

- SQL and procedural SQL extensions.
- PL/SQL for Oracle Database.
- PL/pgSQL for PostgreSQL.
- T-SQL for Microsoft SQL Server.
- Database connectivity libraries for Python, Java, PHP, and other languages.

Important: SQL syntax and programming features differ between database systems.

2. Main Components of Database Programming

The main topics in database programming include:

1. Stored procedures
2. Functions
3. Triggers
4. Cursors
5. Database connectivity
6. Exception handling and transactions

Each component has a different purpose.

Component	Main purpose
Stored procedure	Performs a defined task or series of operations

Function	Calculates or returns a value
Trigger	Executes automatically when a specified database event occurs
Cursor	Processes a query's result rows individually
Database connectivity	Allows an application to communicate with a database

3. Stored Procedures

Definition

A stored procedure is a named collection of SQL statements and programming logic stored in the database and executed when called.

A procedure can perform multiple operations, such as inserting records, updating data, validating information, or generating reports.

Why Do We Use Stored Procedures?

- To reuse database logic.
- To reduce repeated SQL code.
- To centralize business rules.
- To control access to database operations.
- To simplify complex database tasks.

Example: Stored Procedure in MySQL

Suppose we have a table named Student.

```
CREATE TABLE Student (
   StudentID INT PRIMARY KEY,
   Name VARCHAR(100),
   Marks INT
);
```

We can create a procedure to display students whose marks exceed a specified value.

```
DELIMITER //

CREATE PROCEDURE GetStudentsByMarks(
    IN min_marks INT
)
BEGIN
    SELECT StudentID, Name, Marks
    FROM Student
    WHERE Marks > min_marks;
END //

DELIMITER ;
```

To execute the procedure:

```
CALL GetStudentsByMarks(80);
```

This returns students whose marks are greater than 80.

Understanding the Code

- CREATE PROCEDURE defines a procedure.
- GetStudentsByMarks is the procedure's name.
- IN min_marks INT defines an input parameter.
- SELECT retrieves the matching students.
- CALL executes the procedure in MySQL.

Types of Procedure Parameters

Parameters allow information to pass between the caller and the procedure.

Parameter	Meaning
IN	Receives a value from the caller
OUT	Returns a value to the caller
INOUT	Receives a value and can return a modified value

These parameter modes are supported by MySQL and several other procedural SQL systems, but exact syntax varies by DBMS.

Advantages of Stored Procedures

1. Reusable code.
2. Easier maintenance of centralized logic.
3. Can reduce network communication by performing several operations on the server.
4. Can improve access control when users receive permission to execute procedures rather than access underlying tables directly.
5. Can simplify complex database operations.

Disadvantages

1. Syntax differs between database systems.
2. Complex procedures can be difficult to debug.
3. Excessive database-side logic can make applications harder to maintain.
4. Performance depends on implementation, query plans, and workload.

4. Functions

Definition

A database function is a named program that performs a calculation or operation and returns a value, or in some DBMSs, a table of values.

Functions are useful when the same calculation or logic is needed repeatedly.

Example: Function to Calculate a Percentage

Suppose a student obtains 450 marks out of 500.

The percentage is:

$$\text{Percentage}= \frac{\text{Obtained Marks}}{\text{Total Marks}}\times100$$

$$=\frac{450}{500}\times100=90\%$$

A MySQL function can perform this calculation.

```
DELIMITER //

CREATE FUNCTION CalculatePercentage(
    obtained INT,
    total INT
)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    IF total <= 0 THEN
        RETURN NULL;
    END IF;

    RETURN (obtained * 100.0) / total;
END //

DELIMITER ;
```

To call the function:

```
SELECT CalculatePercentage(450, 500);
```

Expected result:

90.00

The function returns NULL when the total is zero or negative, preventing division by an invalid total.

Important Parts

- CREATE FUNCTION defines the function.
- obtained and total are input parameters.
- RETURNS DECIMAL(5,2) specifies the return type.
- DETERMINISTIC declares that the same inputs produce the same result, provided the function's behavior satisfies that declaration.
- RETURN supplies the result.

Types of Database Functions

1. Built-in functions: Provided by the DBMS.

Examples:

SELECT UPPER('alia');

SELECT LENGTH('Database');

SELECT CURRENT_DATE;

SELECT AVG(Marks) FROM Student;

These examples use string, date, and aggregate functions.

2. User-defined functions: Created by the database developer for a particular requirement.

Examples include:

- Calculating student percentages.
- Converting temperatures.
- Calculating employee bonuses.

- Formatting or validating data.

Stored Procedure vs. Function

Stored procedure	Function
Usually called to perform a task.	Usually called to calculate or return a value.
May return results through output parameters or result sets.	Has a defined return value or result type.
May perform several database operations.	Often represents a reusable calculation or transformation.
Called using syntax such as CALL in MySQL.	Can often be used inside expressions such as SELECT.

Note: The distinction is not identical in every DBMS. Some systems support table-valued functions, while restrictions on modifying data from functions vary.

5. Triggers

Definition

A trigger is a database program that executes automatically when a specified event occurs on a table or another supported database object.

Triggers are commonly associated with INSERT, UPDATE, and DELETE events.

Why Are Triggers Used?

- To maintain audit logs.
- To record changes to important data.
- To enforce certain business rules.
- To automatically update related information.
- To detect or reject invalid operations.

Example: Recording Changes in Student Marks

Suppose we want to record every change to a student's marks.

First, create an audit table:

```
CREATE TABLE MarksAudit (
    AuditID INT AUTO_INCREMENT PRIMARY KEY,
    StudentID INT,
    OldMarks INT,
    NewMarks INT,
    ChangedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Now create a trigger:

```
DELIMITER //

CREATE TRIGGER RecordMarksChange
AFTER UPDATE ON Student
FOR EACH ROW
BEGIN
    IF NOT (OLD.Marks <=> NEW.Marks) THEN
        INSERT INTO MarksAudit (
            StudentID,
            OldMarks,
            NewMarks
        )
        VALUES (
            OLD.StudentID,
```

```
        OLD.Marks,
        NEW.Marks
    );
  END IF;
END //

DELIMITER ;
```

Update a student's marks:

```
UPDATE Student
SET Marks = 95
WHERE StudentID = 1;
```

If the marks change, the trigger records the previous marks, the new marks, the student ID, and the timestamp in MarksAudit.

Understanding the Trigger

- CREATE TRIGGER creates the trigger.
- AFTER UPDATE means the trigger runs after an update event.
- FOR EACH ROW means the trigger processes each affected row.
- OLD.Marks represents the previous value.
- NEW.Marks represents the updated value.
- INSERT INTO MarksAudit saves the change.

The <=> operator in this example is MySQL's NULL-safe equality operator. It allows the trigger to detect changes correctly when one of the values is NULL.

Types of Triggers

1. BEFORE Trigger

Runs before the specified operation is completed.

Example uses:

- Validate data.
- Reject invalid values.
- Adjust permitted values before saving.

2. AFTER Trigger

Runs after the specified operation.

Example uses:

- Record audit information.
- Update related records.
- Store information about completed changes.

3. INSTEAD OF Trigger

Runs in place of a specified operation, commonly on views in systems such as SQL Server and Oracle.

Its availability and syntax depend on the DBMS.

Advantages of Triggers

1. Automatic execution.
2. Centralized change tracking.
3. Can enforce rules regardless of which application performs the operation.
4. Useful for audit logging.
5. Can help maintain related information.

Disadvantages of Triggers

1. Hidden execution can make debugging difficult.
2. Poorly designed triggers can reduce performance.
3. Triggers can cause cascading operations or unexpected side effects.
4. Trigger syntax varies across DBMS products.

5. Excessive business logic in triggers can make the system difficult to understand.

Important: Triggers are not a substitute for all database constraints. For rules such as unique identifiers, required values, and valid foreign keys, native constraints are generally preferable.

6. Cursors

Definition

A cursor is a database mechanism used to access and process the rows returned by a query, typically one row at a time.

Normally, SQL works with sets of rows. A cursor is useful when a task specifically requires row-by-row processing.

Why Are Cursors Used?

Suppose a university wants to process each student's record individually to generate a customized report.

A cursor can:

1. Retrieve the query's result set.
2. Fetch one row.
3. Perform an operation on that row.
4. Fetch the next row.
5. Continue until no rows remain.

Main Cursor Operations

Operation	Purpose
Declare	Defines the cursor and its query
Open	Executes or opens the cursor's result set
Fetch	Retrieves the next row

Close	Closes the cursor when finished

Some database systems also provide a DEALLOCATE operation to release the cursor definition.

Cursor Example in MySQL

The following stored procedure reads student names one at a time and displays them in a combined result.

```
DELIMITER //

CREATE PROCEDURE ListStudentNames()
BEGIN
   DECLARE finished INT DEFAULT 0;
   DECLARE student_name VARCHAR(100);
   DECLARE names_list TEXT DEFAULT '';

   DECLARE student_cursor CURSOR FOR
      SELECT Name FROM Student;

   DECLARE CONTINUE HANDLER FOR NOT FOUND
      SET finished = 1;

   OPEN student_cursor;

   read_loop: LOOP
      FETCH student_cursor INTO student_name;
```

```
        IF finished = 1 THEN
            LEAVE read_loop;
        END IF;

        SET names_list = CONCAT(
            names_list,
            student_name,
            ', '
        );
    END LOOP;

    CLOSE student_cursor;

    SELECT names_list AS StudentNames;
END //

DELIMITER ;
```

Execute it:

```
CALL ListStudentNames();
```

If the table contains Ali, Sara, and Ahmed, the procedure returns a combined list of their names.

The actual row order is not guaranteed unless the cursor query includes an ORDER BY clause.

How the Cursor Works

1. DECLARE student_cursor defines the query.

2. OPEN begins processing the result set.
3. FETCH retrieves the next student's name.
4. The NOT FOUND handler detects when no more rows are available.
5. LEAVE exits the loop.
6. CLOSE releases the open cursor.

Types of Cursors

1. Implicit Cursor: Created automatically by the DBMS for certain SQL statements.

2. Explicit Cursor: Declared and controlled by the programmer.

Other classifications include forward-only, scrollable, read-only, and updatable cursors, depending on the DBMS.

Advantages

- Allows row-by-row processing.
- Useful for certain procedural tasks.
- Provides control over individual records.

Disadvantages

- Often slower than set-based SQL operations.
- Can consume additional resources.
- Requires careful handling of end-of-data conditions.
- May make code more complex.

Best practice: Prefer set-based operations such as UPDATE, INSERT ... SELECT, and joins when they can accomplish the same task. Use cursors when row-by-row processing is genuinely needed.

7. Database Connectivity

Definition

Database connectivity is the process of establishing communication between an application and a database so that the application can send queries, retrieve results, and modify data.

For example, when a user logs in to a university portal, the application communicates with a database to verify the user's credentials.

Basic Database Connectivity Process

1. User Application

Website, desktop application or mobile app

2. Database Driver / API

Connects the application to the database

3. Database Server

Executes SQL and manages stored data

4. Query Results

Returned to the application for display or further processing

Steps in Database Connectivity

1. Load or configure the driver: Make the required database connector available.
2. Establish a connection: Supply the server address, database name and authentication details.
3. Create a query or command: Define the SQL operation.
4. Execute the query: Send it to the database server.
5. Process the results: Read returned rows or the operation's status.
6. Handle errors: Detect connection or query failures.
7. Close or release resources: Release connections and statements when no longer needed.

Common Database Connectivity Technologies

Technology	Common use
JDBC	Connecting Java applications to databases
ODBC	Standard database connectivity interface for supported drivers
Python database drivers	Connecting Python programs to database systems
PHP PDO / MySQLi	Connecting PHP applications to databases
ADO.NET	Database access in .NET applications

8. Practical Example: Connecting Python to MySQL

Suppose a Python application needs to retrieve student records from MySQL.

First, install the connector:

pip install mysql-connector-python

Then use the following example:

import mysql.connectorconnection = Nonetry: connection = mysql.connector.connect(host="localhost", user="app_user", password="YOUR_PASSWORD", database="University") cursor = connection.cursor() cursor.execute("SELECT StudentID, Name, Marks " "FROM Student WHERE Marks > %s", (80,)) for row in cursor.fetchall(): print(row) cursor.close()except mysql.connector.Error as error: print("Database error:", error)finally: if connection is not None and connection.is_connected(): connection.close()

Explanation

- mysql.connector provides the MySQL connectivity interface.
- connect() establishes a database connection.
- cursor() creates an object for executing SQL statements.
- execute() sends the query to the database.

- fetchall() retrieves the returned rows.
- try and except handle database errors.
- finally ensures that the connection is closed when possible.

The database University, the table Student, and the account app_user must already exist, with appropriate permissions. Replace the placeholder password with a securely managed credential.

Security point: The query uses a parameter placeholder (%s) instead of joining user input directly into the SQL string. Parameterized queries help prevent SQL injection.

9. SQL Injection and Secure Database Programming

Definition

SQL injection is a security vulnerability in which an attacker manipulates an SQL query by supplying malicious input.

It can occur when an application builds SQL statements by directly concatenating untrusted input.

Unsafe Example

username = input("Enter username: ")query = ("SELECT * FROM Users " "WHERE Username = '" + username + "'")

An attacker may enter specially crafted input that changes the meaning of the SQL statement.

Safe Example

username = input("Enter username: ")cursor.execute("SELECT * FROM Users WHERE Username = %s", (username,))

The parameterized version treats the supplied username as a value rather than SQL code.

Security Best Practices

1. Use parameterized queries.

2. Apply the principle of least privilege to database accounts.
3. Validate input according to the application's requirements.
4. Avoid exposing detailed database error messages to end users.
5. Store credentials securely rather than hard-coding them in source code.
6. Use encrypted connections where appropriate.
7. Use transactions for operations that must succeed or fail together.
8. Keep database software and drivers updated.

Important: Input validation is useful, but it does not replace parameterized queries.

10. Exception Handling in Database Programming

Definition

Exception handling is the process of detecting and managing errors that occur during program execution.

Database errors can include:

- Duplicate primary key values.
- Missing records.
- Invalid data types.
- Connection failures.
- Permission errors.
- Violations of database constraints.

Python Example

try: cursor.execute("INSERT INTO Student " "(StudentID, Name, Marks) " "VALUES (%s, %s, %s)", (101, "Ali", 85)) connection.commit()except mysql.connector.Error: connection.rollback() raise

If the database operation fails, rollback() reverses the current uncommitted transaction where supported by the transaction and driver configuration. raise

passes the exception onward so that the application can handle or report it appropriately.

For a larger application, errors should be handled at the appropriate level and recorded without exposing sensitive information.

11. Transactions in Database Programming

Definition

A transaction is a sequence of database operations treated as one logical unit of work.

For example, transferring money between two bank accounts requires:

1. Deducting money from the sender.
2. Adding money to the receiver.

Both operations should succeed together. If one fails, the transaction should be rolled back.

Example

```
START TRANSACTION;

UPDATE Account
SET Balance = Balance - 1000
WHERE AccountID = 1;

UPDATE Account
SET Balance = Balance + 1000
WHERE AccountID = 2;

COMMIT;
```

If an error occurs, the application should handle it and roll back the transaction rather than committing an incomplete transfer.

ROLLBACK;

The transaction-control statements and error-handling syntax depend on the DBMS and application driver.

Main Transaction Commands

- START TRANSACTION or BEGIN: Starts a transaction.
- COMMIT: Makes the transaction's changes permanent.
- ROLLBACK: Cancels uncommitted changes.
- SAVEPOINT: Creates a point within a transaction to which the transaction can roll back.

12. Stored Procedures, Functions, Triggers and Cursors: Comparison

Feature	Procedure	Function	Trigger	Cursor
Purpose	Perform a task	Return a result	React to a database event	Process result rows individually
Execution	Called explicitly	Called in an expression or through supported syntax	Runs automatically	Controlled by program logic
Typical use	Insert or update records	Calculate a percentage	Audit changes	Process each row
Returns a value	Not necessarily	Yes, by definition	Not typically called for a return value	Fetches rows

Common concern	Complex logic and maintenance	DBMS restrictions	Hidden side effects	Row-by-row performance

13. Best Practices for Database Programming

1. Use clear and meaningful names for tables, functions, procedures and variables.
2. Use parameters instead of hard-coded values.
3. Prefer set-based SQL when it can replace row-by-row processing.
4. Use transactions for related operations that must be atomic.
5. Handle exceptions and database failures.
6. Apply appropriate permissions to users and applications.
7. Use indexes where justified by query patterns and measured performance.
8. Keep database credentials secure.
9. Document complex procedures and triggers.
10. Test database code with valid, invalid, boundary and unexpected inputs.
11. Avoid unnecessary triggers that make behavior difficult to follow.
12. Close connections, cursors and other resources appropriately.

14. Important Exam Questions

Short Questions

1. Define database programming.
2. What is a stored procedure?
3. Differentiate between a stored procedure and a function.
4. Define a database trigger.
5. What are BEFORE and AFTER triggers?
6. What is a cursor?

7. List the basic operations performed on a cursor.
8. Define database connectivity.
9. What are JDBC and ODBC?
10. What is SQL injection?
11. Why are parameterized queries important?
12. Define exception handling in database programming.

Long Questions

1. Explain stored procedures with syntax and a suitable example.
2. Explain database functions and compare them with stored procedures.
3. Describe triggers, their types, advantages and disadvantages.
4. Explain cursors and their operations with an example.
5. Describe the database connectivity process and explain JDBC, ODBC and Python database drivers.
6. Explain how to connect a Python application to MySQL.
7. Discuss secure database programming practices, including SQL injection prevention.
8. Explain the role of transactions and exception handling in database applications.

Topic 08: Database Integrity and Security

Database Management System (DBMS) — BS Computer Science, 7th Semester

1. Introduction to Database Integrity and Security

Database integrity and security are essential concepts in Database Management Systems (DBMS). They ensure that data remains accurate, consistent, reliable, and protected against unauthorized access or modification.

Database Integrity: Ensures that data is correct, valid, and consistent according to defined rules.

Database Security: Protects data from unauthorized access, disclosure, modification, destruction, and other threats.

Real-World Example

Consider a university database containing student records.

Database integrity ensures that:

- Every student has a valid student ID.
- Marks remain within an acceptable range.
- Each student's department exists in the department table.
- A student ID is not duplicated.

Database security ensures that:

- Students can view only the information they are authorized to access.
- Teachers can update marks only when permitted.
- Administrators can manage user accounts.
- Unauthorized users cannot access confidential records.

Important: Integrity protects the correctness of data, while security protects data and database operations from unauthorized or inappropriate access.

2. Database Integrity

Definition

Database integrity refers to the accuracy, validity, and consistency of data stored in a database throughout its lifecycle.

Integrity rules prevent invalid data from being inserted, updated, or deleted in ways that violate the database's requirements.

Types of Database Integrity

1. Entity Integrity

Each table row must be uniquely identifiable. A primary key must be unique and cannot be NULL.

2. Referential Integrity

Foreign-key relationships must refer to valid rows in the referenced table, subject to the database's foreign-key rules.

3. Domain Integrity

Values must follow the permitted data type, range, format, or set of values.

4. User-Defined Integrity

Additional business rules ensure that data follows an organization's specific requirements.

2.1 Entity Integrity

Entity integrity ensures that every row can be uniquely identified through a primary key.

Example:

```
CREATE TABLE Student (
   StudentID INT PRIMARY KEY,
   Name VARCHAR(100) NOT NULL,
   Marks INT
);
```

Here:

- StudentID is the primary key.
- Two students cannot have the same StudentID.
- StudentID cannot be NULL.
- Name cannot be NULL because of the NOT NULL constraint.

If two students have the same student ID, the database rejects the conflicting insertion.

2.2 Referential Integrity

Referential integrity maintains valid relationships between tables.

Suppose a university has a Department table and a Student table.

```
CREATE TABLE Department (
   DepartmentID INT PRIMARY KEY,
   DepartmentName VARCHAR(100) NOT NULL
);
CREATE TABLE Student (
   StudentID INT PRIMARY KEY,
   Name VARCHAR(100) NOT NULL,
   DepartmentID INT,
   FOREIGN KEY (DepartmentID)
      REFERENCES Department(DepartmentID)
);
```

A student can be assigned a department only if the referenced department exists, or NULL if the foreign-key column permits it.

For example, if department ID 10 does not exist, inserting a student with DepartmentID = 10 will fail under normal enforced foreign-key behavior.

2.3 Domain Integrity

Domain integrity ensures that values meet the requirements of their fields.

For example, student marks should be between 0 and 100.

```
CREATE TABLE ExamResult (
   StudentID INT PRIMARY KEY,
   Marks INT CHECK (Marks BETWEEN 0 AND 100)
);
```

The CHECK constraint rejects values outside the permitted range in DBMSs that enforce this constraint.

Other domain rules can include:

- Data types such as INT, DATE, and VARCHAR.
- Allowed values, such as Pass or Fail.
- Required fields.
- Minimum and maximum values.

2.4 User-Defined Integrity

User-defined integrity represents business rules specific to an organization.

Examples:

- An employee's salary must be positive.
- A student cannot register for more than a specified number of credit hours.
- A library member cannot borrow more than five books at a time.

Some rules can be enforced through CHECK constraints or other declarative constraints. More complex rules may require transactions, triggers, or application-level logic.

3. Database Constraints

Definition

A database constraint is a rule applied to a table or column to restrict invalid data and maintain database integrity.

Main Types of Constraints

Constraint	Purpose	Example
PRIMARY KEY	Uniquely identifies each row	StudentID INT PRIMARY KEY

FOREIGN KEY	Enforces a relationship between tables	FOREIGN KEY (DeptID) REFERENCES Department(DepartmentID)
UNIQUE	Prevents duplicate values in a constrained column or combination	Email VARCHAR(100) UNIQUE
NOT NULL	Prevents a column from containing NULL	Name VARCHAR(100) NOT NULL
CHECK	Enforces a specified condition	CHECK (Marks BETWEEN 0 AND 100)
DEFAULT	Supplies a value when one is omitted	Status VARCHAR(20) DEFAULT 'Active'

Note: A DEFAULT constraint provides a value when an appropriate insert omits the column; it does not, by itself, validate every possible value supplied.

Complete Example

```
CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(150) UNIQUE,
    Salary DECIMAL(10,2) CHECK (Salary > 0),
    Status VARCHAR(20) DEFAULT 'Active'
);
```

This table ensures that:

- Each employee has a unique ID.
- The employee's name is required.

- Email addresses cannot be duplicated when non-NULL, subject to DBMS behavior.
- Salary must be positive.
- Status defaults to Active when omitted.

Advantages of Constraints

1. Prevent invalid data.
2. Maintain consistency across applications.
3. Enforce important rules at the database level.
4. Reduce reliance on application-only validation.
5. Improve the reliability of reports and calculations.

4. Database Security

Definition

Database security is the collection of policies, controls, and technologies used to protect databases against unauthorized access, misuse, disclosure, alteration, and destruction.

Objectives of Database Security

Database security commonly follows the CIA triad.

1. Confidentiality

Only authorized people or systems can access sensitive information.

Example: Only authorized staff can view confidential student records.

2. Integrity

Data must not be altered or destroyed improperly.

Example: An unauthorized user must not be able to change examination marks.

3. Availability

Authorized users should be able to access data and services when required.

Example: A university's result portal should remain accessible during result announcements.

Security also includes auditing, accountability, backup protection, and monitoring.

5. Authentication

Definition

Authentication is the process of verifying the identity of a user, application, or system attempting to access a database.

In simple words, authentication answers:

“Who are you?”

Authentication Methods

1. Password-based authentication

The user supplies a username and password.

2. Multi-factor authentication (MFA)

The user provides two or more different types of verification, such as a password and an authenticator-generated code.

3. Certificate-based authentication

A digital certificate helps establish the identity of a client or system.

4. Integrated authentication

The database uses an established identity system, such as an organization's centralized authentication service.

Example

A database login might require a username and password:

```
-- Illustrative SQL Server example

CREATE LOGIN app_user

WITH PASSWORD = 'Use-A-Strong-Secret-Here';
```

This is an administrative example, not a recommended way to embed real credentials in application code. Actual account provisioning should follow the organization's security policy.

Best Practices for Authentication

- Use strong, unique credentials.
- Enable MFA where supported.
- Never store plaintext passwords in an application database.
- Store application passwords using an appropriate salted password-hashing algorithm.
- Apply account lockout or rate-limiting controls where appropriate.
- Avoid sharing database accounts among unrelated users or applications.

Important: Authentication confirms identity; it does not automatically grant permission to perform every database operation.

6. Authorization

Definition

Authorization determines which operations an authenticated user is permitted to perform.

It answers:

“What are you allowed to do?”

For example, a teacher might be allowed to view and update marks, while a student may only view their own results.

Common Database Permissions

- SELECT — read data.
- INSERT — add records.
- UPDATE — modify existing records.
- DELETE — remove records.

- EXECUTE — execute permitted procedures or functions where supported.
- CREATE — create database objects, subject to the DBMS and specific privilege.

SQL Example

The following is a common SQL privilege pattern:

```
GRANT SELECT
ON University.Student
TO student_user;
```

This grants SELECT permission on the specified table to student_user in systems supporting this syntax.

To remove the permission:

```
REVOKE SELECT
ON University.Student
FROM student_user;
```

Exact syntax and object naming vary between DBMS products.

Authentication vs. Authorization

Authentication	Authorization
Verifies identity.	Determines permitted actions.
Answers “Who are you?”	Answers “What can you do?”
Example: Logging in with a password.	Example: Permission to read student marks.
Usually occurs before access is granted.	Uses identity and policy to determine access.

7. Access Control Models

Access control defines how database permissions are assigned and enforced.

7.1 Discretionary Access Control (DAC)

Under discretionary access control, resource owners or authorized administrators can grant permissions to other users.

Example: A database owner grants a colleague permission to read a particular table.

Advantage: Flexible and convenient.

Limitation: Permissions can become difficult to manage in large environments.

7.2 Mandatory Access Control (MAC)

Mandatory access control uses centrally defined policies, often involving security classifications and clearance levels.

Example: A government database may classify records as confidential or highly restricted.

Users can access information only when the applicable policy permits it.

Advantage: Supports strict centralized security policies.

Limitation: Can be less flexible to administer.

7.3 Role-Based Access Control (RBAC)

Role-based access control assigns permissions to roles, and users receive access through their assigned roles.

Example roles:

- Student
- Teacher
- Accountant
- DatabaseAdministrator

Instead of assigning every permission individually, an administrator can assign the appropriate role.

SQL example:

```
CREATE ROLE teacher_role;


GRANT SELECT, UPDATE
ON University.Student
TO teacher_role;
```

A DBMS supporting this syntax allows the role to receive the listed permissions on the specified object. Granting a role to a user requires the relevant role-assignment syntax.

Advantages:

1. Easier permission management.
2. Supports least privilege.
3. Simplifies onboarding and offboarding.
4. Reduces repetitive permission assignments.

7.4 Attribute-Based Access Control (ABAC)

Attribute-based access control makes decisions using attributes associated with users, resources, actions, and the surrounding context.

Example: A teacher may access records only for courses assigned to that teacher and only during an approved period.

ABAC is flexible but requires well-defined policies and reliable attribute information.

8. Principle of Least Privilege

Definition

The principle of least privilege states that each user or application should receive only the minimum permissions needed to perform its assigned tasks.

Example

A reporting application that only displays student records generally needs SELECT permission, not unrestricted permission to delete records or change database structures.

A typical security policy might grant:

- Read-only access to reporting users.
- Limited update access to authorized teachers.
- Account-management permissions to designated administrators.
- Full administrative privileges only to trusted database administrators who need them.

Why is this important?

If an account is compromised, limiting its permissions can reduce the amount of damage an attacker can cause.

9. SQL Injection

Definition

SQL injection is a vulnerability in which untrusted input changes the intended meaning of an SQL statement.

It can expose private information, bypass login checks, modify records, or delete data, depending on the application's permissions and the vulnerable query.

Unsafe Example

Consider a login application that constructs a query using string concatenation:

username = input("Username: ")password = input("Password: ")query = ("SELECT * FROM Users WHERE Username = '" + username + "' AND Password = '" + password + "'")

This is unsafe because specially crafted input may alter the SQL query's structure.

Safe Example: Parameterized Query

cursor.execute("SELECT UserID, Username, PasswordHash " "FROM Users WHERE Username = %s", (username,))

The application should then verify the supplied password against the stored password hash using an appropriate password-verification function.

This approach avoids constructing SQL by directly inserting untrusted values into the query string.

SQL Injection Prevention Techniques

1. Use parameterized queries or prepared statements.
2. Do not concatenate untrusted input into SQL.
3. Apply least privilege to application database accounts.
4. Validate input according to business requirements.
5. Use safe database APIs and drivers.
6. Avoid exposing detailed SQL errors to users.
7. Keep application libraries and database software updated.
8. Store passwords using secure salted password hashing.
9. Conduct authorized security testing.

Important: Escaping characters manually is not a reliable substitute for parameterized queries.

10. Database Auditing and Monitoring

Definition

Database auditing is the process of recording and examining database activities to support accountability, security investigations, and compliance.

Auditing can record:

- Successful and failed login attempts.
- Changes to important records.
- Privilege and role changes.
- Access to sensitive tables.

- Administrative operations.
- Unusual query activity.

Example: Audit Table

```
CREATE TABLE AuditLog (
   AuditID INT PRIMARY KEY,
   UserName VARCHAR(100),
   ActionType VARCHAR(50),
   ActionTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

An application or an appropriately configured database auditing mechanism can record relevant events in this table or another protected logging system.

A table alone does not automatically capture database activity; suitable application logic, triggers, or native auditing facilities are needed.

Benefits of Auditing

1. Helps investigate unauthorized changes.
2. Supports accountability.
3. Helps identify suspicious behavior.
4. Supports compliance with applicable policies and regulations.
5. Provides a record of important operations.

Security recommendation: Audit records should be protected against unauthorized modification and deletion. Sensitive values such as passwords and authentication tokens should not be recorded in logs.

11. Database Backup and Security

Database security includes protecting information against accidental deletion, hardware failures, ransomware, and other incidents.

Important Protection Methods

1. Regular Backups

Create backups according to the importance and recovery requirements of the database.

2. Encryption

Use appropriate encryption for data in transit and, where required, data at rest.

3. Access Restrictions

Limit access to production databases and backup files.

4. Recovery Testing

Regularly test whether backups can actually restore the required data.

5. Monitoring

Monitor database activity, failed logins, unusual access patterns, and administrative changes.

6. Patch Management

Apply security updates to supported database software and related components.

A backup is not sufficient if an attacker can also delete or encrypt it. Protected or immutable backup copies can improve resilience.

12. Database Integrity vs. Database Security

Database integrity	**Database security**
Focuses on data correctness and consistency.	Focuses on protecting data and database operations.
Uses constraints and validation rules.	Uses authentication, authorization, encryption and access control.
Prevents invalid relationships and values.	Prevents unauthorized access and changes.

Example: Marks must be between 0 and 100.	Example: Only authorized teachers can update marks.
Supports trustworthy data.	Supports confidentiality, integrity and availability.

These concepts overlap. For example, authorization helps prevent unauthorized changes that could compromise data integrity.

13. Complete Practical Example: Securing a University Database

Suppose a university stores students, departments, marks, and user accounts.

A good design should include the following controls.

Step 1: Enforce Data Integrity

```
CREATE TABLE Department (
   DepartmentID INT PRIMARY KEY,
   DepartmentName VARCHAR(100) NOT NULL UNIQUE
);
CREATE TABLE Student (
   StudentID INT PRIMARY KEY,
   Name VARCHAR(100) NOT NULL,
   DepartmentID INT,
   Marks INT CHECK (Marks BETWEEN 0 AND 100),
   FOREIGN KEY (DepartmentID)
      REFERENCES Department(DepartmentID)
);
```

These constraints help prevent duplicate identifiers, missing names, invalid marks, and references to nonexistent departments.

Step 2: Create Appropriate Permissions

```
CREATE ROLE result_viewer;


GRANT SELECT

ON University.Student

TO result_viewer;
```

A reporting role receives read permission on the student table instead of unrestricted access.

Step 3: Protect Application Queries

Use parameterized queries in the university website to avoid SQL injection.

Step 4: Audit Sensitive Changes

Record authorized changes to student marks through an appropriate auditing mechanism.

Step 5: Prepare for Recovery

Maintain protected backups and test the recovery process periodically.

Step 6: Monitor Access

Review suspicious login attempts and unexpected changes to important records.

Together, these measures improve data correctness, confidentiality, accountability, and availability.

14. Best Practices for Database Integrity and Security

1. Define primary keys and foreign keys properly.
2. Apply NOT NULL, UNIQUE, and CHECK constraints where appropriate.
3. Use transactions for related operations.
4. Require authentication for database access.
5. Use role-based permissions where suitable.
6. Follow the principle of least privilege.

7. Use parameterized SQL queries.
8. Encrypt sensitive data and connections as appropriate.
9. Protect database credentials and backup files.
10. Enable suitable auditing and monitoring.
11. Keep database software patched.
12. Test backups and recovery procedures.
13. Remove access when a user no longer requires it.
14. Review permissions and security policies periodically.

15. Important Exam Questions

Short Questions

1. Define database integrity.
2. Define database security.
3. What is entity integrity?
4. Explain referential integrity with an example.
5. What is domain integrity?
6. What is a database constraint?
7. Differentiate between a primary key and a foreign key.
8. Define authentication and authorization.
9. What is the CIA triad?
10. Define role-based access control.
11. What is the principle of least privilege?
12. Define SQL injection.
13. What is database auditing?
14. Why are database backups important?

Long Questions

1. Explain the types of database integrity with SQL examples.
2. Describe the different types of SQL constraints and their purposes.
3. Explain database security and the CIA triad.
4. Differentiate between authentication and authorization.
5. Discuss DAC, MAC, RBAC, and ABAC access control models.
6. Explain SQL injection, its risks, and prevention techniques.
7. Discuss database auditing, monitoring, backup protection, and encryption.
8. Explain how to design a secure university database using integrity constraints and access controls.

Step 1: Database create karein

```
CREATE DATABASE CollegeDB;


USE CollegeDB;
```

Samjhein:

- CREATE DATABASE — naya database banata hai.
- USE — batata hai ke ab kis database par kaam karna hai.

Step 2: Department table banayein

```
CREATE TABLE Department (
   DeptID INT PRIMARY KEY AUTO_INCREMENT,
   DeptName VARCHAR(50) NOT NULL UNIQUE,
   Location VARCHAR(50)
);
```

Har cheez ka kaam:

SQL	Purpose
DeptID	Department ki unique ID
PRIMARY KEY	Har department ko uniquely identify karti hai
AUTO_INCREMENT	ID automatically 1, 2, 3... hoti hai
NOT NULL	Department name khali nahi ho sakta
UNIQUE	Do departments ka name same nahi ho sakta

Step 3: Student table banayein

```
CREATE TABLE Student (
   StudentID INT PRIMARY KEY AUTO_INCREMENT,
   StudentName VARCHAR(50) NOT NULL,
   Age INT,
   Gender VARCHAR(10),
   Marks DECIMAL(5,2),
   DeptID INT,

   FOREIGN KEY (DeptID)
      REFERENCES Department(DeptID)
);
```

Yahan do important keys hain:

- Primary Key: StudentID — har student ki apni unique ID.
- Foreign Key: DeptID — student ko Department table ke department ke saath connect karti hai.

Misal ke taur par, agar DeptID = 1 Computer Science department hai, to student table mein DeptID = 1 wala student usi department se belong karega.

Department Table

DeptID (Primary Key)

DeptName · Location

Department.DeptID = Student.DeptID

One department → Many students

Student Table

StudentID (Primary Key)

StudentName · Age · Gender · Marks

DeptID (Foreign Key)

Step 4: Department table mein data insert karein

INSERT INTO Department (DeptName, Location)

VALUES

('Computer Science', 'Block A'),

('Electrical', 'Block B'),

('Islamiyat', 'Block C'),

('Mechanical', 'Block D'),

('Software Engineering', 'Block E');

Check karne ke liye:

SELECT * FROM Department;

Expected data:

DeptID	DeptName	Location
1	Computer Science	Block A
2	Electrical	Block B

3	Islamiyat	Block C
4	Mechanical	Block D
5	Software Engineering	Block E

AUTO_INCREMENT ki wajah se IDs automatically generate hongi.

Step 5: Student table mein data insert karein

```
INSERT INTO Student
(StudentName, Age, Gender, Marks, DeptID)
VALUES
('Ali', 20, 'Male', 85, 1),
('Ayesha', 21, 'Female', 92, 1),
('Ahmed', 22, 'Male', 76, 2),
('Fatima', 20, 'Female', 88, 3),
('Usman', 23, 'Male', 67, 4),
('Zainab', 21, 'Female', 95, 5),
('Hassan', 22, 'Male', 81, 2),
('Maryam', 20, 'Female', 73, 3);
```

Check:

```
SELECT * FROM Student;
```

Foreign Key ka practical rule: Student insert karte waqt DeptID Department table mein pehle se exist karni chahiye. Agar DeptID = 99 exist nahi karti, to foreign key constraint ki wajah se insert fail hoga.

Part 2: SELECT queries

Step 6: Basic data show karna

```
-- Tamam departments
```

```
SELECT * FROM Department;


-- Tamam students
SELECT * FROM Student;


-- Sirf student names
SELECT StudentName FROM Student;


-- Name aur marks
SELECT StudentName, Marks FROM Student;


-- Unique genders
SELECT DISTINCT Gender FROM Student;
```

SELECT data read karne ke liye use hota hai. DISTINCT duplicate values ko ek baar show karta hai.

Step 7: WHERE condition aur operators

```
-- Marks 80 se zyada
SELECT * FROM Student
WHERE Marks > 80;


-- Marks 80 ya zyada
SELECT * FROM Student
WHERE Marks >= 80;


-- Age 20 wale students
```

```
SELECT * FROM Student
WHERE Age = 20;

-- Marks 70 se 90 ke darmiyan
SELECT * FROM Student
WHERE Marks BETWEEN 70 AND 90;

-- Computer Science ya Electrical department IDs
SELECT * FROM Student
WHERE DeptID IN (1, 2);

-- 80 se zyada marks aur age 22 se kam
SELECT * FROM Student
WHERE Marks > 80 AND Age < 22;

-- Marks 70 se kam ya 90 se zyada
SELECT * FROM Student
WHERE Marks < 70 OR Marks > 90;

-- 80 marks ke barabar nahi
SELECT * FROM Student
WHERE Marks <> 80;
```

Step 8: LIKE operator

```
-- A se shuru hone wale names
```

```
SELECT * FROM Student
WHERE StudentName LIKE 'A%';

-- a par khatam hone wale names
SELECT * FROM Student
WHERE StudentName LIKE '%a';

-- Name mein 'an' ho
SELECT * FROM Student
WHERE StudentName LIKE '%an%';
```

- 'A%' — A se start.
- '%a' — a par end.
- '%an%' — name ke andar an.

Step 9: ORDER BY aur LIMIT

```
-- Marks ascending order mein
SELECT * FROM Student
ORDER BY Marks ASC;

-- Marks descending order mein
SELECT * FROM Student
ORDER BY Marks DESC;

-- Top 3 students
SELECT * FROM Student
```

```
ORDER BY Marks DESC
LIMIT 3;


-- Pehle age, phir marks ke hisaab se
SELECT * FROM Student
ORDER BY Age ASC, Marks DESC;
```

Part 3: JOIN — dono tables ko connect karna

Ye aapke practical ka bohat important part hai.

Step 10: INNER JOIN

```
SELECT
   Student.StudentID,
   Student.StudentName,
   Student.Marks,
   Department.DeptName
FROM Student
INNER JOIN Department
ON Student.DeptID = Department.DeptID;
```

Result ka concept:

StudentName	Marks	DeptName
Ali	85	Computer Science
Ayesha	92	Computer Science
Ahmed	76	Electrical
Fatima	88	Islamiyat

Usman 67 Mechanical

JOIN ka matlab: Student table ki DeptID ko Department table ki DeptID ke saath match karke department ka naam bhi show karna.

Step 11: LEFT JOIN

```
SELECT
   Department.DeptName,
   Student.StudentName
FROM Department
LEFT JOIN Student
ON Department.DeptID = Student.DeptID;
```

LEFT JOIN Department table ke tamam departments show karega, chahe kisi department mein koi student na bhi ho. Aise department ke student columns mein NULL aayega.

Step 12: JOIN ke saath WHERE

```
-- Sirf Computer Science ke students
SELECT Student.StudentName, Student.Marks,
      Department.DeptName
FROM Student
INNER JOIN Department
ON Student.DeptID = Department.DeptID
WHERE Department.DeptName = 'Computer Science';


-- 80 se zyada marks wale students aur unke departments
SELECT Student.StudentName, Student.Marks,
      Department.DeptName
```

```
FROM Student
INNER JOIN Department
ON Student.DeptID = Department.DeptID
WHERE Student.Marks > 80;
```

Part 4: Aggregate functions

Step 13: COUNT, SUM, AVG, MAX, MIN

```
-- Total students
SELECT COUNT(*) AS TotalStudents
FROM Student;

-- Average marks
SELECT AVG(Marks) AS AverageMarks
FROM Student;

-- Highest marks
SELECT MAX(Marks) AS HighestMarks
FROM Student;

-- Lowest marks
SELECT MIN(Marks) AS LowestMarks
FROM Student;

-- Total marks
SELECT SUM(Marks) AS TotalMarks
```

```
FROM Student;
```

AS result ke column ko temporary readable naam deta hai.

Step 14: GROUP BY

Har department mein kitne students hain?

```
SELECT
   Department.DeptName,
   COUNT(Student.StudentID) AS TotalStudents
FROM Department
LEFT JOIN Student
ON Department.DeptID = Student.DeptID
GROUP BY Department.DeptID, Department.DeptName;
```

Har department ke average marks:

```
SELECT
   Department.DeptName,
   AVG(Student.Marks) AS AverageMarks
FROM Department
INNER JOIN Student
ON Department.DeptID = Student.DeptID
GROUP BY Department.DeptID, Department.DeptName;
```

Step 15: HAVING

WHERE individual rows ko filter karta hai, jabke HAVING groups ko filter karta hai.

```
SELECT
   DeptID,
   COUNT(*) AS TotalStudents
```

FROM Student

GROUP BY DeptID

HAVING COUNT(*) > 1;

Ye sirf un departments ko show karega jin mein ek se zyada students hain.

Part 5: UPDATE, DELETE aur ALTER

Step 16: UPDATE

-- Ali ke marks update karna

UPDATE Student

SET Marks = 90

WHERE StudentName = 'Ali';

-- Ayesha ki age update karna

UPDATE Student

SET Age = 22

WHERE StudentName = 'Ayesha';

-- Department ki location change karna

UPDATE Department

SET Location = 'Block F'

WHERE DeptID = 1;

Important: WHERE zaroor lagayein. Agar UPDATE Student SET Marks = 90; likhen, to har student ke marks 90 ho jayenge.

Step 17: DELETE

-- Student ID 5 delete karna

DELETE FROM Student

WHERE StudentID = 5;

-- Marks 40 se kam wale students delete karna

DELETE FROM Student

WHERE Marks < 40;

Agar kisi department mein students linked hain, to us department ko delete karne par foreign key restriction aa sakti hai. Pehle related records handle karne honge.

Step 18: ALTER TABLE

-- Student table mein email column add karna

ALTER TABLE Student

ADD Email VARCHAR(100);

-- Column ka data type change karna

ALTER TABLE Student

MODIFY StudentName VARCHAR(100);

-- Column ka naam change karna

ALTER TABLE Student

RENAME COLUMN Email TO StudentEmail;

ALTER TABLE existing table ki structure change karta hai.

Part 6: Subqueries

Step 19: Average se zyada marks

SELECT StudentName, Marks

FROM Student

WHERE Marks > (

```
    SELECT AVG(Marks)
    FROM Student
);
```

Pehle inner query average marks calculate karegi; phir outer query average se zyada marks walay students show karegi.

Step 20: Highest marks wala student

```
SELECT StudentName, Marks
FROM Student
WHERE Marks = (
    SELECT MAX(Marks)
    FROM Student
);
```

Step 21: Computer Science department ke students

```
SELECT StudentName
FROM Student
WHERE DeptID = (
    SELECT DeptID
    FROM Department
    WHERE DeptName = 'Computer Science'
);
```

Part 7: Keys aur constraints ka extra practical

Step 22: UNIQUE, CHECK aur DEFAULT

Aap ek additional table bana kar constraints practice kar sakti hain:

```
CREATE TABLE Teacher (
    TeacherID INT PRIMARY KEY AUTO_INCREMENT,
```

```
    TeacherName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Salary DECIMAL(10,2) DEFAULT 30000,
    Age INT CHECK (Age >= 18),
    DeptID INT,
    FOREIGN KEY (DeptID)
        REFERENCES Department(DeptID)
);
```

Is mein:

- PRIMARY KEY — teacher ki unique ID.
- FOREIGN KEY — teacher ko department se link karti hai.
- UNIQUE — email duplicate nahi ho sakti.
- NOT NULL — teacher name required hai.
- DEFAULT — salary na dene par default 30000.
- CHECK — age kam az kam 18 honi chahiye. CHECK enforcement MySQL version par depend kar sakti hai.

Sample insert:

```
INSERT INTO Teacher (TeacherName, Email, Age, DeptID)
VALUES ('Mr. Ahmed', 'ahmed@example.com', 35, 1);
```

Salary mention nahi ki, is liye default salary apply hogi.

Part 8: Views aur Indexes

Step 23: VIEW

```
CREATE VIEW StudentDepartmentView AS
SELECT
    Student.StudentID,
```

```
    Student.StudentName,
    Student.Marks,
    Department.DeptName
FROM Student
INNER JOIN Department
ON Student.DeptID = Department.DeptID;
```

View ko run karein:

```
SELECT * FROM StudentDepartmentView;
```

VIEW ek saved query hoti hai jo data ko table jaisi form mein show karti hai.

Step 24: INDEX

```
CREATE INDEX idx_student_name
ON Student(StudentName);


CREATE INDEX idx_student_marks
ON Student(Marks);
```

Index searching aur filtering ko tez kar sakta hai, khaas taur par large tables mein. Lekin indexes extra storage lete hain aur insert/update ko kuch slow kar sakte hain.

Part 9: Transactions

Step 25: COMMIT aur ROLLBACK

```
START TRANSACTION;


UPDATE Student
SET Marks = 95
WHERE StudentID = 1;
```

```
SELECT * FROM Student
WHERE StudentID = 1;
```

```
COMMIT;
```

COMMIT changes ko save karta hai.

Agar changes cancel karne hon, aur abhi COMMIT na kiya ho:

```
START TRANSACTION;

UPDATE Student
SET Marks = 10
WHERE StudentID = 1;

ROLLBACK;
```

ROLLBACK transaction ke uncommitted changes ko undo karta hai. MySQL mein ye behavior transactional storage engine, jaise InnoDB, par depend karta hai.